SYSTEM ARCHITECTURE & INTERVIEW GUIDE

NyayaSetu

An LLM-powered legal and RTI document simplifier for English and Marathi: how the four services fit together, how data flows between them, and how every feature works.

Final-year major project · Report version 1.0 · October 2026

Stack: React · Spring Boot · FastAPI · PostgreSQL + pgvector · Docker Compose

Source: github.com/Abhay5474/Nyayasetu (branch `claude/modest-brown-996d6q`)

What this report is for

It explains the system the way you would explain it in a viva or a technical interview: the problem, the architecture, the reason behind each design decision, the data flow between services, the problems that were hit and fixed, the measured results, and an honest list of limitations. Each major feature ends with likely interview questions and model answers.

1 · Frontend
React 18 SPA behind nginx; English and Marathi UI

2 · Backend
Spring Boot gateway: auth, storage, orchestration, alerts

3 · AI service
FastAPI: OCR, RAG, LLM, chat, draft generator

4 · PostgreSQL
App data plus pgvector knowledge base

Contents

1	Problem, goals and scope	3
2	Architecture at a glance	5
3	Technology choices and why	7
4	Service 1 — Frontend (React + nginx)	8
5	Service 2 — Backend (Spring Boot gateway)	11
6	Service 3 — AI service (FastAPI)	16
7	Service 4 — PostgreSQL and pgvector	21
8	The legal knowledge base and RAG in depth	23
9	End-to-end data flows	26
10	Functionality walk-through (interview view)	31
11	Security and privacy	35
12	Deployment and operations	37
13	Testing and evaluation (with real results)	39
14	Problems faced and how they were solved	42
15	Limitations and future work	44
16	Interview question bank	45
A	Appendix — API reference, configuration, glossary	49

HOW TO READ THE NUMBERS IN THIS REPORT

Figures such as test counts, chunk counts, defaults and evaluation scores come from the code and from runs made during development. Where a figure was measured on real models (the retrieval evaluation) it is labelled. Where something could not be tested here (for example a specific third-party LLM) the report says so.

1 Problem, goals and scope

Most people who receive an RTI reply, a legal notice or a court order cannot read it. The language is technical, the document is often a scan or a phone photo, and the deadline that matters (usually 30 days) is buried in a paragraph. Missing it can mean losing the right to appeal.

1.1 The problem in concrete terms

- **Language barrier.** Official text is dense English or formal Marathi. A citizen needs plain language in the language they think in.
- **Format barrier.** Documents arrive as digital PDFs, scanned pages, or photographs taken on a phone, sometimes with broken Marathi text layers.
- **Time-bound consequences.** The Right to Information Act, 2005 gives 30 days for a first appeal (Section 19(1)) and 90 days for a second appeal (Section 19(3)). A missed date is expensive.
- **Trust.** A summary that invents a section number or states an outdated fee does harm. The system must ground its statements in real law and show where each statement comes from.

1.2 What NyayaSetu does

Capability	What the user gets
Understand	Upload a PDF or photo. The system extracts the text (OCR when needed), checks it against a legal knowledge base, and returns a plain-language summary, key points, deadlines, required actions and legal references with source citations, in English or Marathi.
Ask	A continuous, multi-turn chat about the uploaded document. Answers are grounded in the document text and the legal knowledge base, with verbatim quotes and a clear “not answerable from this document” signal.
Track	Deadlines found in documents are stored per user, grouped by urgency, and can trigger e-mail and SMS reminders 7, 3 and 1 day before and on the day.
Act	A draft generator produces a ready-to-file RTI application, first appeal or reply to a fee demand in English or Marathi as PDF or Word, with checks such as the computed last date to appeal.
Trust	Every citation is labelled “Official text”, “Government reprint” or “Unverified summary”, with the date the text is current as of, the verification status and reviewer notes. Superseded rules are never used.

1.3 Design goals

1. **Correctness over fluency.** Ground every statement in the document or in cited law; refuse to guess dates.
2. **Works with imperfect inputs.** Scans, photos, broken text layers, mixed English and Marathi.
3. **Privacy by default.** Each user sees only their own documents; drafts are not stored; secrets stay server-side.
4. **Operable by a student on a laptop.** One command (`docker compose up`) starts everything; offline modes exist for tests.
5. **Honest about limits.** The product explains documents; it does not give legal advice, and says so in the UI and in every prompt.

1.4 Scope boundaries

The knowledge base focuses on the RTI Act 2005, the Maharashtra RTI Rules (2005 and the 2026 notifications), the Central fee rules, the Consumer Protection Act 2019, plus summaries of a few other laws and judgments. It is a major-project prototype: the legal texts are not human-verified, the Marathi RTI Act text is OCR-derived, and Section 15 lists what a production system would still need.

INTERVIEW FRAMING

“It is a retrieval-augmented generation system with a product layer around it. The LLM is the smallest part of the codebase. Most of the engineering is about getting trustworthy text in (OCR, cleaning), trustworthy law in (knowledge base with provenance), and trustworthy output out (schema validation, grounding, deadlines, drafts).”

2 Architecture at a glance

NyayaSetu runs as **four containers** defined in `docker-compose.yml` : a web front end, a Java gateway, a Python AI service and a PostgreSQL database. Only the front end is published to the host (port 3000). Everything else is reachable only inside the Compose network.

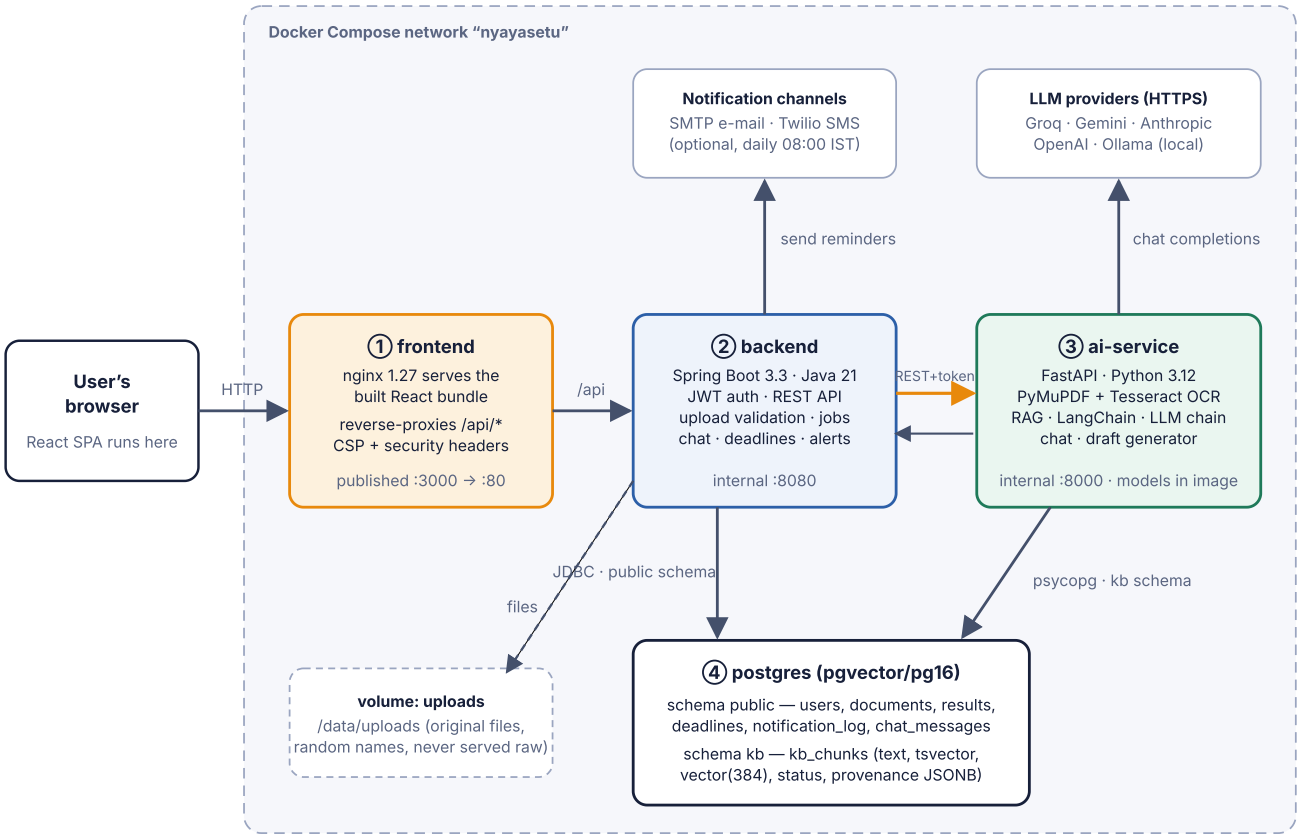


Figure 2.1 — The four services, their ports and the three kinds of storage (relational data, vectors, uploaded files).

2.1 Responsibilities in one table

Service	Owns	Deliberately does not do
① **frontend**	UI, language switch, routing, polling for job status, rendering results, drafts preview and downloads, token storage in the browser	Any legal logic, any secret, any direct call to the AI service or the database
② **backend**	Identity (register/login/JWT), authorisation (who owns which document), upload validation and storage, job orchestration, persistence of results/deadlines/chat, reminders, error translation	OCR, retrieval, prompts, LLM calls (it only calls the AI service)

Service	Owns	Deliberately does not do
③ **ai-service**	Text extraction and OCR, cleaning, chunking, knowledge-base ingestion and hybrid retrieval, prompts, LLM calls with fallbacks, JSON validation, grounding, document chat, draft templates and rendering	User accounts, file ownership, long-term storage of user content (it is stateless for user data)
④ **postgres**	Durable storage for application data (schema `public`) and the vector/keyword knowledge base (schema `kb`)	Business logic (only constraints, indexes and the retrieval queries issued by the AI service)

2.2 Why this split (the interview answer)

- **Separation of concerns by skill set.** Java/Spring is strong at authentication, transactions and long-lived APIs. Python is where the OCR, embedding and LLM libraries live. Putting each in the language of its ecosystem avoids bridging hacks.
- **Trust boundary.** The browser talks only to the gateway. The AI service is never exposed: it trusts a shared secret header, and only the backend holds it. A bug in the AI service cannot be reached by an anonymous user.
- **Independent scaling and failure.** OCR and LLM calls are slow and bursty; the gateway keeps serving login, history and deadlines while the AI service is busy or restarting (the Compose file lets the backend start without waiting for the AI service to be healthy).
- **Testability.** The backend tests mock the AI client; the AI service tests use an in-memory store, a hash embedder and a mock LLM; both also have real-database tests.

2.3 The request path in three sentences

The browser sends every API call to nginx, which proxies `/api/*` to Spring Boot. Spring Boot authenticates the JWT, checks ownership, does the work it owns (storage, database) and, for anything involving understanding text, calls the AI service with an internal token. The AI service reads or writes the knowledge base in PostgreSQL, calls an LLM provider if needed, and returns JSON; Spring Boot stores what matters and answers the browser.

3 Technology choices and why

Layer	Choice	Why this and not the alternative
UI	React 18, Vite 5, Tailwind CSS 3, Axios, react-router, lucide-react	Component model suits a stateful results page and chat; Vite gives fast builds; Tailwind keeps styling consistent without a design system; Axios interceptors centralise the auth header and 401 handling.
Web server	nginx 1.27 (alpine)	Serves static files efficiently, adds security headers/CSP, and proxies `/api` so the browser sees one origin (no CORS in production).
Gateway	Spring Boot 3.3.5, Java 21, Spring Security, Spring Data JPA, Flyway, jjwt 0.12.6, BCrypt(12), RestClient	Mature security and transaction model; Flyway gives versioned schema migrations; RestClient is the modern synchronous HTTP client.
AI service	FastAPI + uvicorn, Pydantic v2, LangChain core, PyMuPDF, Tesseract (mar+hin+eng), fastembed (ONNX), psycopg 3 + pgvector, python-docx	Pydantic gives strict, self-documenting JSON contracts; PyMuPDF is fast and also renders the draft PDFs; Tesseract has Marathi/Hindi models; fastembed runs multilingual embeddings and a cross-encoder without PyTorch (smaller image).
Database	PostgreSQL 16 with pgvector	One database for relational data, full-text search and vectors; avoids running a separate vector database; transactional and easy to back up.
Embeddings	`paraphrase-multilingual-MiniLM-L12-v2` (384 dimensions)	Small, CPU-friendly, covers English/Hindi/Marathi well enough for retrieval.
Reranker	`jina-reranker-v2-base-multilingual` cross-encoder (lexical fallback)	A cross-encoder reads query and passage together and corrects the first-stage ranking; it was the largest single gain in the evaluation.
LLM access	LangChain chat models: Anthropic, OpenAI-compatible (Groq, Gemini, xAI…), Ollama, plus a mock	Provider-agnostic: any OpenAI-compatible endpoint works by changing a URL and key. A fallback chain protects against outages.
Packaging	Docker Compose, GitHub Actions CI	Reproducible start-up and automated tests for all three code bases.

INTERVIEW QUESTION THIS ANSWERS

"Why not do everything in one Python app?" It would be shorter, yes. The split shows a real service boundary: the identity, storage and transaction logic lives in the stack built for it, while the AI-specific work is isolated behind a narrow, token-protected API. It also let the gateway degrade gracefully when the AI side is slow or down.

4 Service 1 — Frontend (React + nginx)

The front end is a single-page application. Its job is to make a slow, multi-step backend feel simple: upload, wait with feedback, read a result, ask follow-up questions, draft a reply.

4.1 Structure of the code

```
frontend/src
├─ App.jsx                routes: / · /login · /register · /app (protected) · 404
├─ pages/                 Landing · AuthPages · Dashboard · DocumentDetail · Deadlines · Drafts · SettingsPage
├─ components/            Layout/Navbar · Dropzone · ChatPanel · StatusBadge · ui.jsx (Alert, Field, Segmented…)
│  └─ results/            Sections (summary, key points, checklist, deadline timeline, references, sources),
│                         Provenance (trust badges), ProcessingPanel (progress while the job runs)
│  └─ drafts/             DraftPaper (on-screen preview of a draft)
├─ context/               AuthContext (session, login, logout, auto-expiry) · I18nContext (en/mr)
├─ api/                   client.js (Axios instance, interceptors) · endpoints.js (typed API modules)
├─ i18n/                  strings.js (UI text en + mr) · drafts.js (draft-form text)
├─ utils/                 format.js (dates, deadline grouping) · draftForms.js (form rules + payload builder)
└─ hooks.js               usePolling (only while the tab is visible)
```

4.2 How the main screens work

DASHBOARD — UPLOAD AND HISTORY

A drag-and-drop zone accepts a PDF or image (client-side type and size hints; the real validation is on the server). The user picks the output language with a segmented control (English / मराठी) and submits. The upload shows a progress bar (Axios `onUploadProgress`), then the document appears in the history list with a status badge. While any document is `UPLOADED` or `PROCESSING`, the list polls every 3 seconds; polling stops when nothing is active and pauses when the browser tab is hidden.

DOCUMENT DETAIL — THE RESULT

Two tabs: **Explanation** and **Ask questions** (the active tab is kept in the URL, `?tab=ask`, so a refresh keeps it). While processing, a progress panel is shown and the page polls every 2.5 seconds. When complete, the explanation is rendered in sections:

- **Summary card** and **key points**;
- **Action checklist** with priorities (high, medium, low);
- **Deadline timeline** — each deadline shows the exact date or the relative period, the original sentence it came from, days remaining, and a toggle to mark it done (this calls the backend and updates the deadlines page);
- **Legal references**, each linked to a numbered source (S1, S2…) with a “Cited” marker;
- **Sources & citations**, each with a trust badge and an expandable provenance panel (section 4.4);
- **Caveats**, where the model lists uncertainty.

The user can switch output language without re-uploading: the backend re-runs only the AI explanation step on the already-extracted text. The original file can be viewed (fetched with the auth header and shown through a blob URL, because a plain link would not carry the JWT), and the page can be printed with a print stylesheet that hides navigation.

CHAT TAB

A message list with “You” and “NyayaSetu” bubbles, suggested follow-up questions returned by the model, an answer-language selector, Enter to send and Shift+Enter for a new line. Each answer shows verbatim quotes from the document and the cited sources; if the model says it cannot answer from the document, that is displayed distinctly rather than as a confident answer. History is loaded from the backend, so a conversation survives a page reload.

DEADLINES PAGE

All deadlines across documents, grouped by `groupDeadlines()` into Overdue, Next 7 days, Later, No fixed date, and Done, each linking back to its source document.

DRAFTS PAGE

Choose a document type, fill a form whose fields depend on the type, preview the generated document on screen (rendered from the server's blocks), read warnings and filing notes, and download PDF or Word. Details are in section 10.7.

SETTINGS

Profile (name, phone, preferred language), e-mail and SMS reminder toggles, and account deletion (which cascades to all of the user's data).

4.3 Cross-cutting front-end design

Concern	Implementation
Authentication state	`AuthContext` keeps `{token, expiresAt, user}` in `localStorage` (guarded by try/catch for private windows). A timer logs the user out at expiry; the Axios response interceptor logs out on any 401 except on `/auth/*`. `ProtectedLayout` redirects anonymous users to `/login` and remembers where they came from.
API layer	One Axios instance with `baseURL: '/api'` and a request interceptor adding `Authorization: Bearer …`. `errorMessage()` converts any failure (timeout, no network, field errors, server message) into a short safe string.
Internationalisation	`I18nContext` exposes `t` (UI strings) and `lang`. The UI language is separate from the language of the AI output, because a user may read the interface in English but want a Marathi explanation. The choice is persisted.
Accessibility	`Field` associates labels with inputs through `useId` and `cloneElement`; the language control uses `role="radiogroup"`; tabs use `role="tablist"`; alerts use `role="alert"` / `status`.
Performance	Hashed assets cached for a year by nginx; the SPA fallback is never cached; polling is conditional on visibility.
Same-origin design	In production the browser only talks to its own origin. nginx forwards `/api/` to the backend preserving the original `Host` header, so Spring's CORS check sees matching Origin and Host. In development, Vite proxies `/api` to port 8080 with `changeOrigin: false` for the same reason.

4.4 Trust badges (a feature worth explaining)

Each retrieved source carries its provenance from the knowledge base. The UI turns that into:

Badge	Meaning	Extra detail shown
Official text	Parsed from a gazette PDF	"Text as of *date*", verification status, gazette reference, OCR warning, notes, link
Government reprint	Official text as reprinted in a government handbook (not the gazette)	Same, plus a note that later amendments may be missing
Unverified summary	A summary written for this project	A warning to confirm before relying on it

4.5 nginx: the other half of the front-end container

- Serves the built bundle from `/usr/share/nginx/html` with `try_files … /index.html` so deep links work.
- Proxies `/api/` to `backend:8080` with 300-second read/send timeouts (uploads plus OCR are slow), request buffering off, and a 26 MB body limit (the application limit is 25 MB).
- Sets `Content-Security-Policy` (only own origin plus Google Fonts), `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, `Referrer-Policy: no-referrer` and a restrictive `Permissions-Policy`.

Q Why store the JWT in localStorage instead of an HttpOnly cookie?

It keeps the API stateless and avoids CSRF entirely (no ambient credentials), which is why CSRF protection is disabled on the backend. The trade-off is exposure to XSS, which is mitigated by a strict Content-Security-Policy (`script-src 'self'`), React's escaping, and no use of `dangerouslySetInnerHTML` . A production hardening step would be short-lived access tokens plus a refresh token in an HttpOnly cookie.

Q Why poll instead of using WebSockets or Server-Sent Events?

A job takes seconds to a minute and the status changes only a few times. Polling every 2.5–3 seconds is simple, works through any proxy and needs no extra server state. The hook polls only while something is active and the tab is visible, so the cost is small. SSE would be the next step if jobs became longer or more numerous.

5 Service 2 — Backend (Spring Boot gateway)

The backend is the only component the browser can reach. It owns identity, ownership of data, safe file handling and the orchestration of long-running work. It contains no OCR, retrieval or prompt logic: for those it calls the AI service.

5.1 Package structure

Package	Role
`auth`	Register and login (`AuthController` , `AuthService` , DTO validation). Returns a JWT plus the user profile.
`security`	`JwtService` (sign/verify HS256), `JwtAuthenticationFilter` , `LoginThrottle` , `RestAuthenticationEntryPoint` (JSON 401/403).
`user`	Profile read/update, account deletion (cascades to all the user's rows and files).
`storage`	`FileStorageService` and `FileType` : upload validation by content, atomic write, SHA-256, safe paths.
`document`	Upload, list, detail, reprocess, results, original file, delete; `DocumentProcessor` (the asynchronous job) and `DocumentStateService` (the status state machine, transactional).
`gateway`	`AiServiceClient` : the only code that talks to the AI service; `AiServiceException` carries a user-safe message and the upstream status.
`chat`	Persisted multi-turn Q&A per document.
`deadline`	Captures deadlines from AI results, lists them, updates their status.
`notification`	`DeadlineAlertScheduler` , `DeadlineAlertService` , e-mail (SMTP) and SMS (Twilio) senders, bilingual message templates, send log.
`draft`	Stateless proxy for the draft generator, with input checks and error mapping.
`web` , `config`	One `GlobalExceptionHandler` producing a uniform error JSON; typed configuration (`AppProperties`), security chain, async executor.

5.2 The REST API

Method	Path	What it does
POST	`/api/auth/register`	Create an account; validates e-mail, 8–72 character password, optional international phone; returns token + user.
POST	`/api/auth/login`	Verify credentials (throttled); returns token + user.
GET · PATCH · DELETE	`/api/users/me`	Read, update (name, phone, language, alert preferences) or delete the account.
POST	`/api/documents`	Multipart upload (`file` , `language`) → stores the file, starts processing, returns the document.
GET	`/api/documents`	Paged list of the user's documents with per-language status.
GET	`/api/documents/{id}`	Document detail and status (this is what the UI polls).
POST	`/api/documents/{id}/process?language=`	Process in another language (or `force=true` to redo); reuses the extracted text.
GET	`/api/documents/{id}/results/{lang}`	The stored AI result (JSON) for one language.

Method	Path	What it does
GET	`/api/documents/{id}/file` · DELETE `/…/{id}`	Stream the original file (owner only) · delete document, results, deadlines, chat and file.
GET · POST · DELETE	`/api/documents/{id}/chat`	History · ask a question · clear the conversation.
GET · PATCH	`/api/deadlines` · `/api/deadlines/{id}`	List (optionally by status) · mark pending/done.
POST	`/api/drafts/preview` · `/api/drafts/file?format=`	Draft generator: JSON preview · PDF or Word file.
GET	`/actuator/health`	Liveness/readiness for Docker; includes a health indicator for the AI service.

5.3 Security design

- **Stateless JWT.** The token (HS256, signed with a secret of at least 32 bytes — the application refuses to start otherwise) carries the user id as subject plus e-mail and role. Default lifetime is 12 hours. `JwtAuthenticationFilter` validates it on every request and places an `AuthUser` in the security context. `SessionCreationPolicy.STATELESS`; no server session exists.
- **CSRF disabled on purpose.** Credentials travel in the `Authorization` header, not in a cookie, so a cross-site form cannot attach them.
- **Passwords.** BCrypt with cost 12. Passwords longer than 72 bytes are rejected rather than silently truncated (BCrypt's limit). E-mail uniqueness is enforced case-insensitively by a unique index on `lower(email)`.
- **Brute-force guard.** `LoginThrottle` allows 5 failures per (e-mail + client IP) within 15 minutes, then returns HTTP 429. It is in-memory and therefore per instance.
- **Ownership by construction.** Repositories expose methods like `findByIdAndUserId`. Requesting someone else's document returns **404**, not 403, so the API does not even reveal that the id exists.
- **Security headers** on API responses: `nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: no-referrer`. CORS allows only configured origins and exposes `Content-Disposition` (needed by the draft download).

5.4 Upload handling (defence in depth)

1. Reject empty files and files above the limit (25 MB by default; checked twice, once from the declared size and once while copying).
2. **Identify the file by its first bytes** (magic numbers): PDF `%PDF-`, PNG, JPEG, TIFF, BMP, WEBP. The client-supplied name and MIME type are never trusted.
3. Generate a random name (`<userId>/<uuid>.<ext>`), resolve the path and verify it stays inside the storage root (path-traversal check).
4. Stream into a temporary file while computing SHA-256, then **atomically move** into place, so a crash never leaves a half-written file.
5. Insert the database row; if that fails, delete the stored file so no orphans remain.

The original filename is sanitised before it is stored for display. The AI service re-checks the file type from bytes as well.

5.5 The document lifecycle

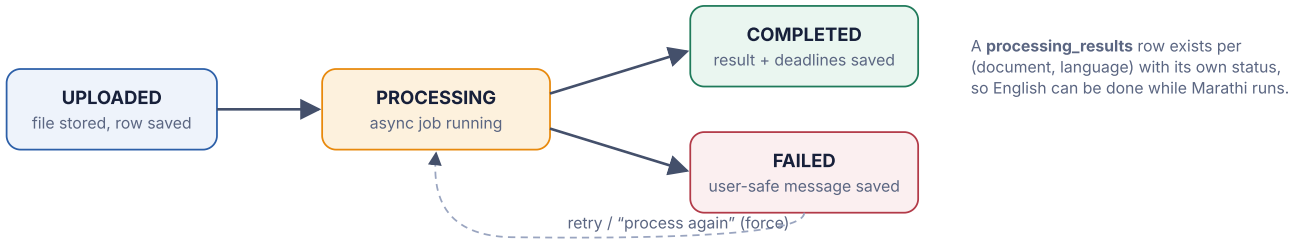


Figure 5.1 — Status flow. `DocumentStatus` *= UPLOADED / PROCESSING / COMPLETED / FAILED; each language result has* `ResultStatus` *PROCESSING / COMPLETED / FAILED.*

HOW THE ASYNCHRONOUS JOB WORKS

- `DocumentService.upload()` stores the file and row, then calls `start()`. `DocumentStateService.beginProcessing()` is a small transactional method that refuses to start a second job for the same (document, language) while one is running, and refuses to redo a completed one unless `force` is set. That makes double-clicks and retries safe (idempotent).
- `DocumentProcessor.process()` runs on the `documentExecutor` thread pool (2 core / 4 max threads, queue of 100). If the queue is full, the request does not hang: the result is marked FAILED with "The server is busy right now". The HTTP request returns immediately with status PROCESSING.
- The job: extract text via the AI service only if it is not already stored, save the extraction (text, page count, method `digital` / `ocr` / `mixed`, detected language), call `/v1/simplify`, save the JSON result and the model name, then capture deadlines.
- **One retry for transient failures** (connection error, 5xx) after 2 seconds; client errors (4xx) and "LLM not configured" (503) are final.
- Deadline capture is wrapped in its own try/catch so a bookkeeping error can never lose a finished explanation. Deadlines are captured only by the first language processed, so translating does not duplicate them.
- `StartupRecovery` runs at start-up and marks any job left in PROCESSING by a crash or restart as FAILED (jobs wait in memory and are lost on restart), so users see a retryable error instead of a spinner that never ends.

5.6 The gateway client to the AI service

- Three `RestClient` instances with different read timeouts: extract (300 s), simplify/chat/drafts (240 s) and health (3 s).
- Every call adds `X-Internal-Token`. The client is forced to **HTTP/1.1**; the JDK client would otherwise try an `h2c` upgrade that uvicorn rejects, which corrupted multipart uploads during development.
- Failures become `AiServiceException`. FastAPI's `detail` text is passed to the user only for safe categories: 422 (for example an unreadable scan, or a draft form field problem) and AI errors tagged `code: llm_*` (bad key, rate limit, overloaded model). Anything else becomes a generic message, so internal details never leak.

5.7 Data model

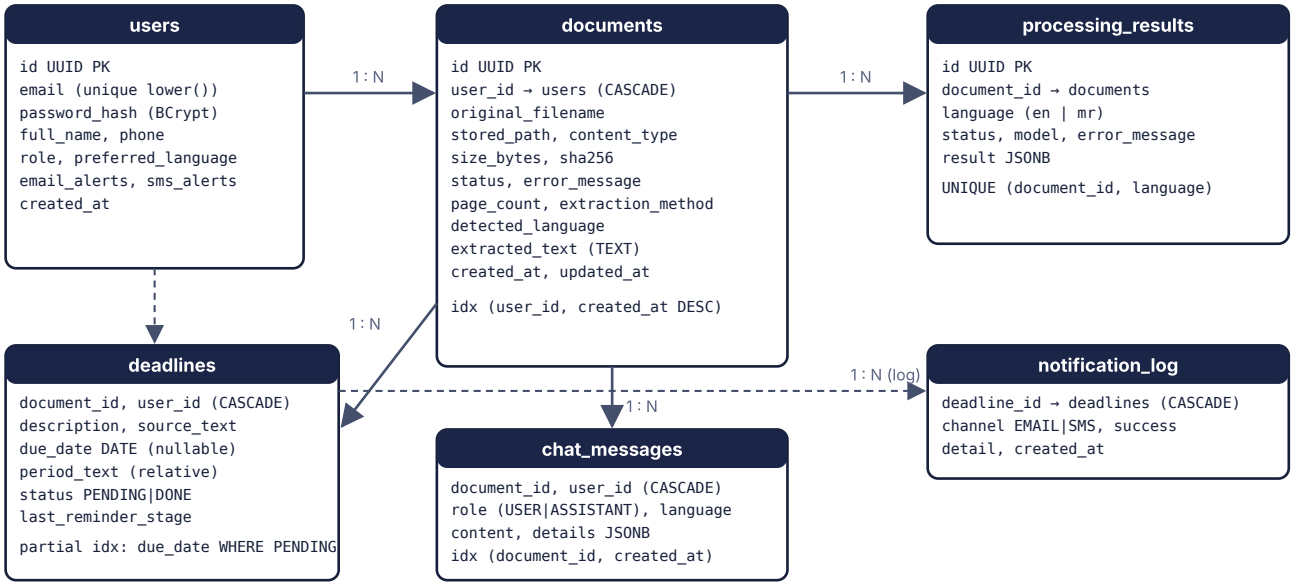


Figure 5.2 — Schema `public` (managed by Flyway migrations V1 and V2). Every child row cascades on delete, so deleting a user or document leaves nothing behind.

Why JSONB for the AI result? The explanation is a document-shaped object (summary, lists of deadlines, references, sources) that the front end consumes whole. Keeping it as JSONB means the AI schema can evolve without a migration, while the fields the backend needs to query (deadlines) are also extracted into a proper relational table with an index.

5.8 Chat persistence

`ChatService.ask()` verifies ownership, refuses if the document has not been read yet (HTTP 409 `NOT_READY`), enforces a limit of 100 questions per document, loads the last 10 messages as history, calls the AI service and then saves the user's question and the assistant's answer. The assistant's message stores the structured extras (answerable flag, quotes, cited sources, follow-ups, model) in the `details` JSONB column. The method is intentionally **not** transactional: an LLM call can take many seconds and must not hold a database connection.

5.9 Deadlines and reminders

Covered with the data flow in section 9.4 and as a feature in section 10.5.

5.10 Error handling

`GlobalExceptionHandler` maps every failure to one JSON shape (`timestamp, status, code, message, fieldErrors`): validation errors list the offending fields, upload size/type errors have stable codes, unknown routes are 404, and any unexpected exception is logged and reported as a generic 500 without a stack trace.

Q What happens if the AI service is down when a user uploads a file?

Upload still succeeds (the file is stored and the row saved). The background job's call fails; after one retry the result is marked FAILED with the message "The AI service is not responding right now. Please try again shortly." The user can press "process again" later. Login, history and deadlines keep working because they never call the AI service.

Q How do you prevent one user reading another user's document?

All reads go through repository methods that include the user id (`findByIdAndUserId`), and a miss is reported as 404. The id comes from the verified JWT, never from the request. There is an integration test that tries to fetch another user's document and expects 404.

6 Service 3 — AI service (FastAPI)

The AI service turns bytes into understanding. It extracts and cleans text, retrieves the relevant law, asks an LLM for a structured explanation, checks that explanation, answers follow-up questions and generates legal drafts. It keeps no user data: everything it needs arrives in the request.

6.1 Endpoints

Method	Path	Purpose
GET	`/health`	Public. Database ping, knowledge-base chunk count and ingestion state, embedding model, reranker, LLM provider and fallback chain, OCR engine.
POST	`/v1/extract`	File → cleaned text, page count, method (`digital` , `ocr` , `mixed`), language hint, warnings.
POST	`/v1/simplify`	Text + language → structured explanation + numbered sources + model name.
POST	`/v1/chat`	Document text + question + history → answer, quotes, sources, follow-ups.
GET · POST	`/v1/kb/search` · `/v1/kb/ingest`	Search the knowledge base · add entries at run time.
GET	`/v1/llm/check`	One tiny call to each configured model; reports which ones answer.
POST	`/v1/drafts/preview` · `/v1/drafts/file`	Draft generator: blocks as JSON · PDF/Word bytes.

All `/v1/*` routes require `X-Internal-Token` , compared with `hmac.compare_digest` (constant-time). The service is not published outside the Compose network.

6.2 Composition root and start-up

`container.py` builds, once per process: the embedder, the vector store (`PgVectorStore` in production, `InMemoryStore` in tests), the hybrid retriever (store + embedder + reranker), the LLM chain, the simplifier, the document chat and the extractor. At start-up the knowledge base is ingested in a **background thread** so the service starts answering health checks immediately; `/health` reports `state: running|ready|failed` . Ingestion is incremental: chunks whose content hash is unchanged are skipped.

Provider modes make the service testable anywhere: `EMBEDDING_PROVIDER=hash` (no model download), `RERANKER=lexical` , `LLM_PROVIDER=mock` (extractive demo output) and `VECTOR_STORE=memory` .

6.3 Text extraction pipeline

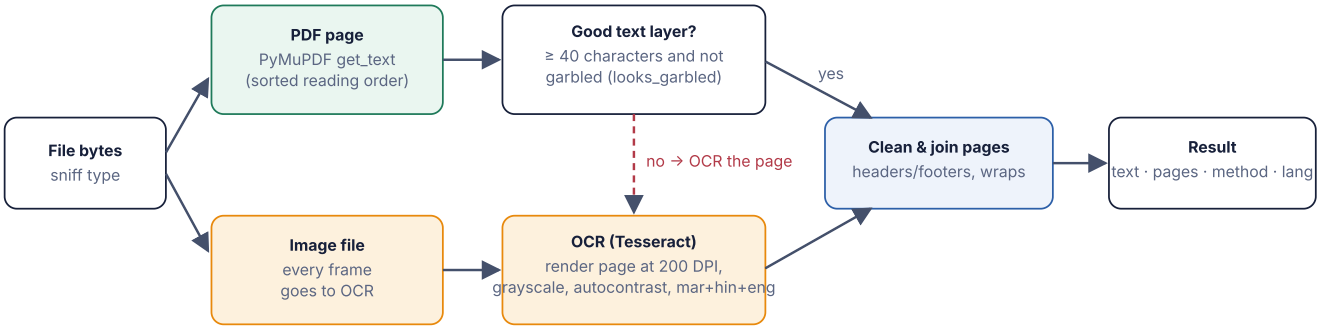


Figure 6.1 — Extraction. The decision is made per page, so a PDF that mixes typed and scanned pages is handled correctly (method `mixed`).

- **Type detection from bytes.** PDF, PNG, JPEG, TIFF, BMP and WEBP; anything else is a 422 with a helpful message. Password-protected or empty PDFs are rejected clearly; very long PDFs are processed up to 60 pages with a warning.
- **Broken Marathi text layers.** Some Maharashtra gazette PDFs contain a text layer in which vowel signs are doubled and the words are unreadable even though the page looks fine. `looks_garbled()` detects this pattern and forces OCR for that page, even though the text layer is "long enough". This was found when ingesting the real gazettes.
- **OCR.** Tesseract with languages `mar+hin+eng` (only the installed packs are used; missing ones are logged, not fatal). Pages are rendered at 200 DPI (300 DPI for knowledge-base ingestion). A light pre-processing step converts to grayscale, applies autocontrast and up-scales very small images. An optional PaddleOCR engine is wired in but untested here.
- **Cleaning** (`processing/cleaning.py`): normalises whitespace, removes running headers and footers (short lines repeated on most pages), merges hard-wrapped lines into paragraphs while keeping list items and headings, and is written to be **Devanagari-safe**. Python's `\w` does not match Devanagari vowel signs, so a custom tokenizer is used wherever words are counted.
- **Language hint** (`en | mr | hi | unknown`) from the ratio of Devanagari characters plus Marathi/Hindi marker words, used by the UI as information (the output language is chosen by the user).
- **Empty result guard.** If fewer than 20 readable characters come out, the user gets "No readable text found. The scan may be too blurry, blank or in an unsupported script."

> **MEASURED**
>
> On three synthetic phone-photo style images (rotation, noise, JPEG compression: an English application, an English reply, a Marathi application) Tesseract recovered 98–100% of the words. These images were generated for testing, not real photographs; real photographs will do worse.

6.4 Explanation pipeline (`/v1/simplify`)

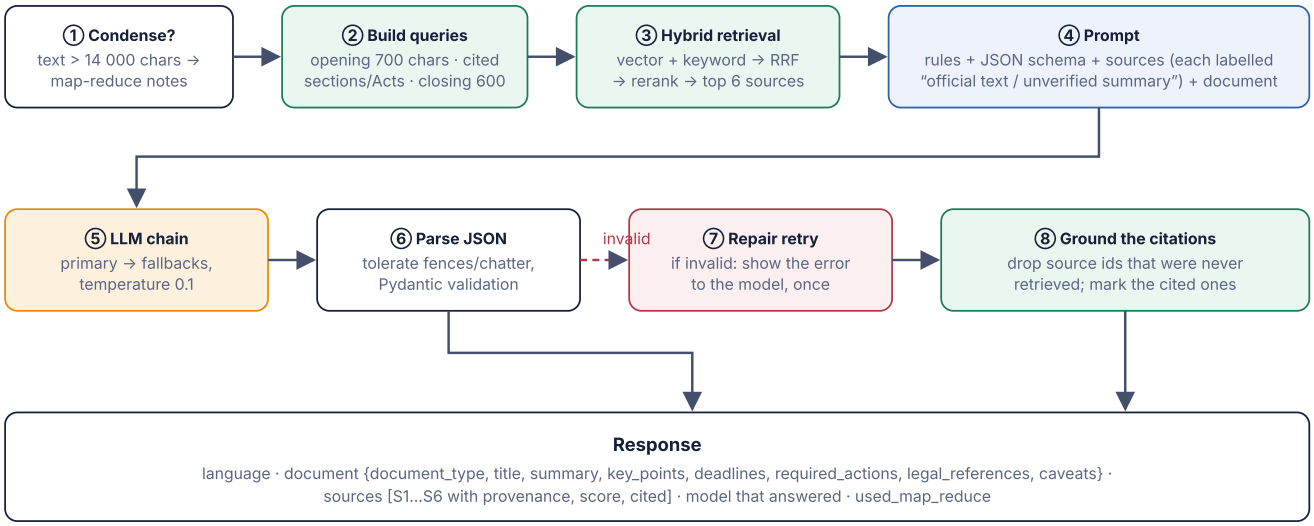


Figure 6.2 — What happens between "text in" and "explanation out".

① MAP-REDUCE FOR LONG DOCUMENTS

Direct prompting is used up to 14 000 characters (`max_direct_chars`). Longer texts are split into chunks of about 5 400 characters with overlap (at most 24 chunks) using a sentence-aware splitter whose separators include the Devanagari danda "।". Each chunk is condensed **in parallel** (at most 4 at a time) into faithful working notes that keep every date, amount, name and citation exactly as written. The notes, not the raw text, go into the final prompt. The response flag `used_map_reduce` tells the UI this happened.

②–③ RETRIEVAL

Several query formulations are retrieved together and fused: the document opening, the statutes/sections the document itself cites (found with regular expressions such as "Section 19(1)", "RTI Act"), and the closing paragraph for long texts. Section 8 explains the retriever itself.

④ THE PROMPT (RULES THE MODEL MUST FOLLOW)

1. Ground every statement in the DOCUMENT or the REFERENCE SOURCES; never invent facts, names, dates, amounts, case numbers or section numbers.
2. Write for a reader with no legal training; explain unavoidable terms in brackets.
3. Write all free text in the requested language (natural Devanagari Marathi, not transliteration) but keep statute names, numbers and proper nouns exact.
4. **Deadlines:** an exact ISO date only when the document states it or it follows unambiguously; otherwise `date: null` plus a relative `period` . Never guess. Today's date is supplied.
5. Sources are labelled *official text* (with "current as of" date, status, OCR flag and reviewer NOTE) or *unverified summary*. Prefer official text; later Rules or notifications prevail over amounts in an Act; the NOTE must be considered.
6. This is an explanation, not legal advice; real uncertainty goes into `caveats` .
7. Reply with one JSON object that matches the schema and nothing else.

⑥–⑦ MAKING THE OUTPUT RELIABLE

LLMs often add code fences or a sentence before the JSON. `extract_json()` finds the first balanced JSON object, and Pydantic validates and normalises it (lenient priorities, string lists, ISO dates parsed or set to null). If parsing or validation fails, the model is shown its own reply plus the error and asked once to answer again with only valid JSON. If that also fails, the request ends in a clear `GenerationError` (HTTP 502) rather than a half-valid result.

⑧ GROUNDING THE CITATIONS

A model can cite "S9" when only six sources exist. After parsing, every `source_id` is checked against the sources actually retrieved; unknown ids are removed with a warning, valid ones are marked `cited`, and cited sources are sorted first. The UI therefore never shows a citation that points at nothing.

6.5 Document chat (`/v1/chat`)

A conversation about *one* document, grounded in that document and in the knowledge base.

- **Follow-up questions retrieve badly alone** ("and what about the second one?"). The retrieval query therefore combines the previous user question with the new one, and the knowledge base is searched with both formulations.
- **Long documents.** If the document is within 14 000 characters it is sent whole. Otherwise a small per-document hybrid index is built in memory (chunks of 900 characters), cached in an LRU of 8 documents so later questions do not re-embed the file, and only the best excerpts (kept in reading order) go into the prompt.
- **History.** The last 8 messages are replayed, each truncated to 1 500 characters, so the model can resolve "it", "that deadline" and so on.
- **Output** (validated like the explanation): `answer`, `answerable` (false when neither the document nor the sources contain the answer), `quotes`, `source_ids` and up to three `follow_ups`.
- **Verified quotes.** The model is asked to quote the document, but quotes are **checked**: only those that really occur in the document text (ignoring whitespace and case, at least 8 characters) are kept, at most two. A quote cannot be invented.
- Only cited sources are returned, again after the id check.

6.6 LLM access, fallbacks and error handling

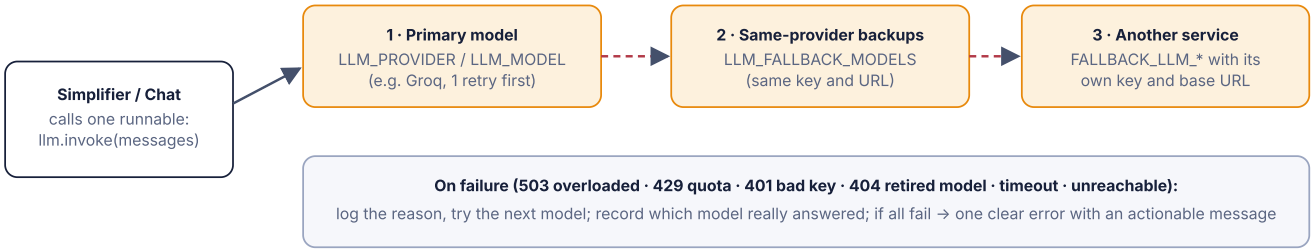


Figure 6.3 — The fallback chain, built with LangChain's `with_fallbacks`, so it works unchanged inside prompt pipelines and batch calls.

- **Providers.** `anthropic`, `openai` (any OpenAI-compatible endpoint: Groq, Google's Gemini endpoint, xAI, OpenRouter…), `ollama` (local) and `mock`. Switching provider is configuration only.
- **Key hygiene.** API keys and URLs are trimmed of quotes, spaces and a pasted "Bearer " prefix; an empty base URL is treated as unset.
- **Actionable errors.** `llm/errors.py` classifies failures (bad key, unknown model, rate limit, overloaded, timeout, unreachable) into short messages that name the setting to check. They travel back as a coded 502/429/503 and the backend shows them to the user, instead of a bare "500".
- **Which model answered** is tracked per thread and stored with the result, so a fallback answer is never mislabelled as coming from the primary.
- **Self-diagnosis.** `python -m app.llm.check` and `GET /v1/llm/check` probe every model in the chain in parallel with a 30-second limit; `--list-models` prints the model names the configured keys can use (model names are retired often).
- **Mock mode** (`LLM_PROVIDER=mock`) produces a deterministic extractive summary (dates found by regex, key sentences) so the whole system can be demonstrated and tested without any API key. It does not translate or truly simplify, and the documentation says so.

6.7 The draft generator (preview)

A separate module (`app/drafts/`) that does **not** use the LLM: Pydantic models validate the form, deterministic English/Marathi templates build a neutral list of blocks, and two renderers produce the file (PDF with PyMuPDF's HTML engine, which shapes Devanagari correctly; Word with python-docx). Section 10.7 explains it in full.

6.8 Source layout

```
ai-service/app
├─ main.py            routes, exception handlers (draft input errors, LLM call errors)
├─ container.py       composition root · background KB ingestion
├─ config.py          typed settings from environment (keys cleaned)
├─ security.py        internal-token dependency
├─ schemas.py         Pydantic contracts (requests, SimplifiedDocument, Source, KbEntry, chat)
├─ extraction/        extractor (PDF/image logic) · ocr (Tesseract, optional Paddle)
├─ processing/        cleaning (headers, wraps, language hint, garble check) · chunking · Devanagari tokenizer
├─ rag/               embeddings · store (pgvector/in-memory) · retriever (RRF) · reranker · ingest
│                     legal_parser (Act/Rules/gazette splitting) · sources (manifest → entries)
├─ llm/               providers (fallback chain) · prompts · simplifier · chat · errors · check · mock_generator
├─ drafts/            schemas · templates (en/mr) · render (PDF, Word)
└─ evaluation/        dataset · metrics · runner · CLI (python -m app.evaluation)
ai-service/knowledge_base   manifest, source PDFs, proofreadable extracted text, seed JSON
ai-service/eval             questions.jsonl (60 labelled questions) and results
```

Q Why does the model output JSON instead of free text?

The product needs structure: deadlines must become rows, actions a checklist, references links to numbered sources. A schema also lets us validate and repair the answer and drop bad citations. The cost is occasional invalid JSON, handled by tolerant extraction and one repair retry.

Q How do you reduce hallucination here?

Six layers: (1) a prompt that forbids invented facts and dates; (2) retrieval of real legal text with provenance; (3) sources labelled by trust level and an instruction to prefer official text; (4) schema validation with a repair step; (5) post-hoc grounding that removes citations to sources that were never retrieved, and chat quotes that are not in the document; (6) a visible UI with source badges and caveats so the user can verify. It does not eliminate errors, which is why every page says it is not legal advice.

Q What if the document is 200 pages?

Extraction is capped at 60 pages with a warning. Anything over 14 000 characters is condensed with parallel map-reduce into notes first. For chat, a per-document index retrieves only the relevant excerpts for each question.

7 Service 4 — PostgreSQL and pgvector

One PostgreSQL 16 instance (image `pgvector/pgvector:pg16`) holds everything that must survive a restart. It is shared by two services that never touch each other's tables, which is why the data is split into two schemas.

7.1 Two schemas, two owners

Schema	Owner	Contents and lifecycle
`public`	Backend (Flyway)	users, documents, processing_results, deadlines, notification_log, chat_messages, plus Flyway's own history table. Created by versioned migrations `V1__init.sql` and `V2__chat.sql`.
`kb`	AI service	`kb.kb_chunks`: the legal knowledge base. Created and upgraded in code (`CREATE TABLE IF NOT EXISTS`, `ALTER TABLE … ADD COLUMN IF NOT EXISTS`) so a new version of the service upgrades an existing database in place.

A REAL BUG THIS DESIGN FIXED

The knowledge-base table was first created in `public`. Flyway then refused to start with "Found non-empty schema(s) `public` but no schema history table", because the AI service had created a table before Flyway ran. The fix was the dedicated `kb` schema, which makes start-up order irrelevant.

`db/init.sql` runs once when the volume is first created and executes `CREATE EXTENSION IF NOT EXISTS vector`. The AI service also issues this statement defensively. Data lives in the `pgdata` named volume; `docker compose down` keeps it, `down -v` deletes it.

7.2 The knowledge-base table

```
kb.kb_chunks
  id            TEXT PRIMARY KEY          -- "<kb_id>#<chunk number>"
  kb_id         TEXT NOT NULL             -- the provision, e.g. rti-act-2005-en-s19
  kind          TEXT                      -- act | rule | notification | judgment | other
  title, citation, jurisdiction, language, text
  content_hash  TEXT                      -- sha256 of text + provenance: skip unchanged chunks on re-ingest
  status        TEXT DEFAULT 'in_force'   -- in_force | superseded | repealed
  meta          JSONB                     -- provenance: source_type, gazette_ref, version_date, verified_at/by,
ocr, notes, origin
  embedding     vector(384)               -- paraphrase-multilingual-MiniLM-L12-v2
  tsv           tsvector GENERATED ALWAYS AS (to_tsvector('simple', title||' '||citation||' '||text)) STORED
  updated_at    TIMESTAMPTZ

  GIN  index on tsv                       -- keyword search
  HNSW index on embedding (vector_cosine_ops)   -- approximate nearest neighbour
```

- **Why a generated `tsvector` column?** The keyword index can never drift from the text; the database maintains it on every write.
- **Why the `simple` text-search configuration?** It does no stemming or stop-word removal for a specific language, so it behaves the same for English and Marathi. (Postgres ships no Marathi dictionary.)
- **Why HNSW?** Fast approximate nearest-neighbour search with good recall and no training step; cosine distance matches how the embedding model was trained.
- **Why is status a column, not a deletion?** Superseded law (for example the Maharashtra Rules of 12 June 2026, replaced on 7 July 2026) is kept for history but excluded by every retrieval query with `status NOT IN ('superseded','repealed')`.

- **Origin tag and pruning.** Chunks ingested from files are tagged `origin: files`; after a directory ingest, chunks of that origin whose entry no longer exists are deleted. Chunks added through the API (`origin: api`) are never pruned.

7.3 The two retrieval queries

VECTOR SEARCH

```
SELECT … , 1 - (embedding <=> :query_vec) AS sim
FROM kb.kb_chunks
WHERE status NOT IN ('superseded','repealed')
ORDER BY embedding <=> :query_vec
LIMIT 20;
```

KEYWORD SEARCH WITH RARITY WEIGHTING

The query is turned into OR-ed terms (long legal questions would match nothing if every word had to appear). Postgres' built-in `ts_rank_cd` weighs all words equally, so filler words like "how" and "days" decided the ranking. The evaluation exposed this (section 13). The replacement computes, for each term, an **inverse document frequency** from `ts_stat` (cached until the next write) and scores each chunk as the sum of weights of the terms it contains:

```
score(chunk) = Σ  idf(term) · [chunk contains term],     idf(t) = ln(1 + N / df(t))
SELECT …, ( CASE WHEN tsv @@ to_tsquery('simple', :t1) THEN :w1 ELSE 0 END + … ) AS score
FROM kb.kb_chunks WHERE tsv @@ to_tsquery('simple', 't1 | t2 | …') AND status NOT IN (…)
ORDER BY score DESC LIMIT 20;
```

7.4 Connection handling and operations

- The backend uses HikariCP via Spring; the AI service uses a `psycopg_pool` pool (1–5 connections) with the pgvector type registered on each connection.
- Healthcheck: `pg_isready`; the AI service and backend wait for `service_healthy` before starting. The backend does *not* wait for the AI service to be healthy, only started, so login and history stay available while models load.
- Backup: `pg_dump` of the database covers both schemas (the knowledge base can also be rebuilt from the repository at any time, which is why losing `kb` is harmless but losing `public` is not).

Q Why pgvector instead of a dedicated vector database like Pinecone or Qdrant?

At this scale (hundreds of chunks) a separate system is unnecessary operational weight. Keeping vectors beside the relational data gives one backup, one connection pool, transactional upserts, and lets keyword (full-text) and vector search run in the same database. If the corpus grew to millions of chunks, a dedicated engine or partitioning would be worth discussing.

Q What is the difference between the two schemas and why does it matter?

Ownership. Flyway owns `public` and demands it is empty on first run; the AI service owns `kb` and manages it in code. Separate schemas remove the start-up race I hit and make it obvious which service may change which table.

8 The legal knowledge base and RAG in depth

Retrieval-augmented generation means the model is shown the relevant law at question time instead of being trusted to remember it. Nothing is trained or fine-tuned. The quality of the whole product is bounded by two things: what is in the knowledge base, and whether retrieval finds it.

8.1 What is in the knowledge base

Source	Status	Type	Entries and notes
Maharashtra RTI Rules, notification of **7 July 2026** (English + Marathi)	in force	Official text	Rules 1–5 + annexures (forms A, B, C). Fees ₹10 application, ₹2 per page, ₹20 appeal; first appeal 30 days, second appeal 90 days.
Maharashtra RTI Rules, 12 June 2026 (en + mr)	superseded	Official text	27 rules (fees ₹30 / ₹5 / ₹100). Kept for history; never retrieved.
Maharashtra RTI Rules, 2005	superseded	Official text	5 rules + annexures.
RTI Act 2005, Marathi text as of 1 Dec 2014	in force	Official text (OCR)	82 paragraph-level entries; section numbers are unreliable after OCR, so no section labels are attached.
RTI Act 2005, English	in force	Government reprint	Sections 1–31 + Second Schedule, from a 2008 handbook; later amendments (e.g. 2019) not reflected, stated in a note.
Central RTI fee rules 2005 + 2005/2006 amendments	in force	Government reprint	Rules 1–5 and the amendment notifications; note says Maharashtra authorities follow the Maharashtra Rules.
Consumer Protection Act, 2019	in force	Official text	107 sections as enacted; note that the 2021 Jurisdiction Rules changed the money limits.
Seed summaries (JSON)	unverified	Summary	NI Act s.138–142, Limitation Act, BNS/BNSS provisions, Legal Services Authorities Act s.12, Central RTI Rules 2012, RTI online portal, four Supreme Court judgments, a few Marathi summaries.

At the time of the last run the knowledge base held **318 entries stored as 467 chunks**.

8.2 From a gazette PDF to a retrievable, citable entry



Figure 8.1 — Manifest-driven ingestion of official sources.

- **Manifest.** Each source records: title, kind, language, jurisdiction, **status**, **version_date**, gazette reference, parser, expected provision count, notes, verification fields (`verified_at` , `verified_by` , currently empty) and a `source_type` .
- **Proofreadable middle step.** Extracted text is written to a plain file the team can read and correct (OCR confuses Devanagari digits, for example "१९९." for "११."). The corrected file, not the PDF, is what is ingested. Several such corrections were made by hand while loading the real gazettes.

- **Parsers.** `numbered` : rule headings with a dash (“3. Procedure.–”), with strict sequential numbering so a stray “7.” inside a sentence is not a new rule. `numbered_margin` : Acts whose marginal notes are interleaved in the text. `paragraphs` : noisy scans where numbers cannot be trusted. `single` : a schedule or amendment notification kept whole.
- **Safety net.** `expected_provisions` turns a silent mis-parse into a loud error: “parsed 72 provisions, expected 107” stopped ingestion until the parser was fixed.
- **Provenance on every chunk**: source type, title, gazette reference, version date, verification, OCR flag and notes. The notes carry caveats such as “later Rules prevail over these amounts”, and the prompt instructs the model to consider them.

8.3 Retrieval: hybrid search with reranking

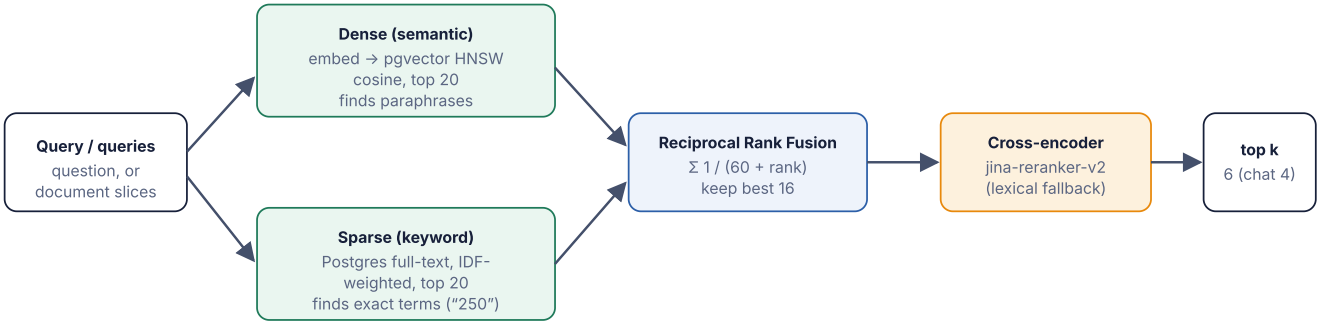


Figure 8.2 — Both retrievers only see `in_force` chunks. With several queries, all ranked lists are fused together.

Stage	What it contributes
Dense retrieval	Understands meaning across wording and languages (“How long do I have to appeal?” ≈ “within thirty days … first appeal”). Weak on exact numbers and rare terms.
Keyword retrieval	Exact terms: section numbers, amounts, names. Weak on paraphrase.
RRF fusion	Combines two ranked lists without needing their scores to be comparable: a document ranked well in both rises to the top.
Cross-encoder rerank	Reads query and passage *together* and judges relevance more precisely than a vector comparison. Costly, so only 16 candidates are reranked. If the model cannot load, a lexical reranker (term and bigram overlap) takes over.
Status filter	Superseded/repealed law is excluded in SQL, so a fee that has since changed cannot be quoted as current.

8.4 The trust model in one paragraph

Three kinds of source, shown to the user as badges and to the model as labels: **official text** (from a gazette), **government reprint** (from an official handbook, possibly stale) and **unverified summary** (written for the project). The model is told to prefer official text and to treat a NOTE as binding context. The user sees the same distinctions, the date each text is current as of, and whether a human has verified it. Verification fields exist in the manifest but are still empty: this is a known gap (section 15).

Q What is the difference between RAG and fine-tuning, and why RAG here?

Fine-tuning changes the model’s weights; RAG supplies knowledge at query time. Law changes (the Maharashtra fees changed twice in 2026), must be citable, and must show its source. RAG allows updating the knowledge base without retraining, makes each statement traceable to a provision, and lets superseded rules be switched off with a status flag.

Q Why hybrid search rather than just embeddings?

Legal questions often hinge on exact tokens: “Section 8(1)(j)”, “₹250 per day”, “30 days”. Embeddings blur such details; keyword search finds them but misses paraphrases and cross-language matches. Fusing both and reranking was measurably better than either alone (section 13).

Q How do you make sure an outdated rule is not quoted?

Every chunk has a `status`. Retrieval queries exclude superseded and repealed chunks. Fee questions are tested to return the 7 July 2026 Rules and never the 12 June 2026 ones, and the evaluation reports “superseded text returned: 0%”.

9 End-to-end data flows

Six flows cover everything the system does. Each diagram is a sequence: time runs downward, solid arrows are calls, dashed arrows are replies, and a yellow box is a note. The orange numbers show the order of the calls.

9.1 Upload and explanation (the core flow)

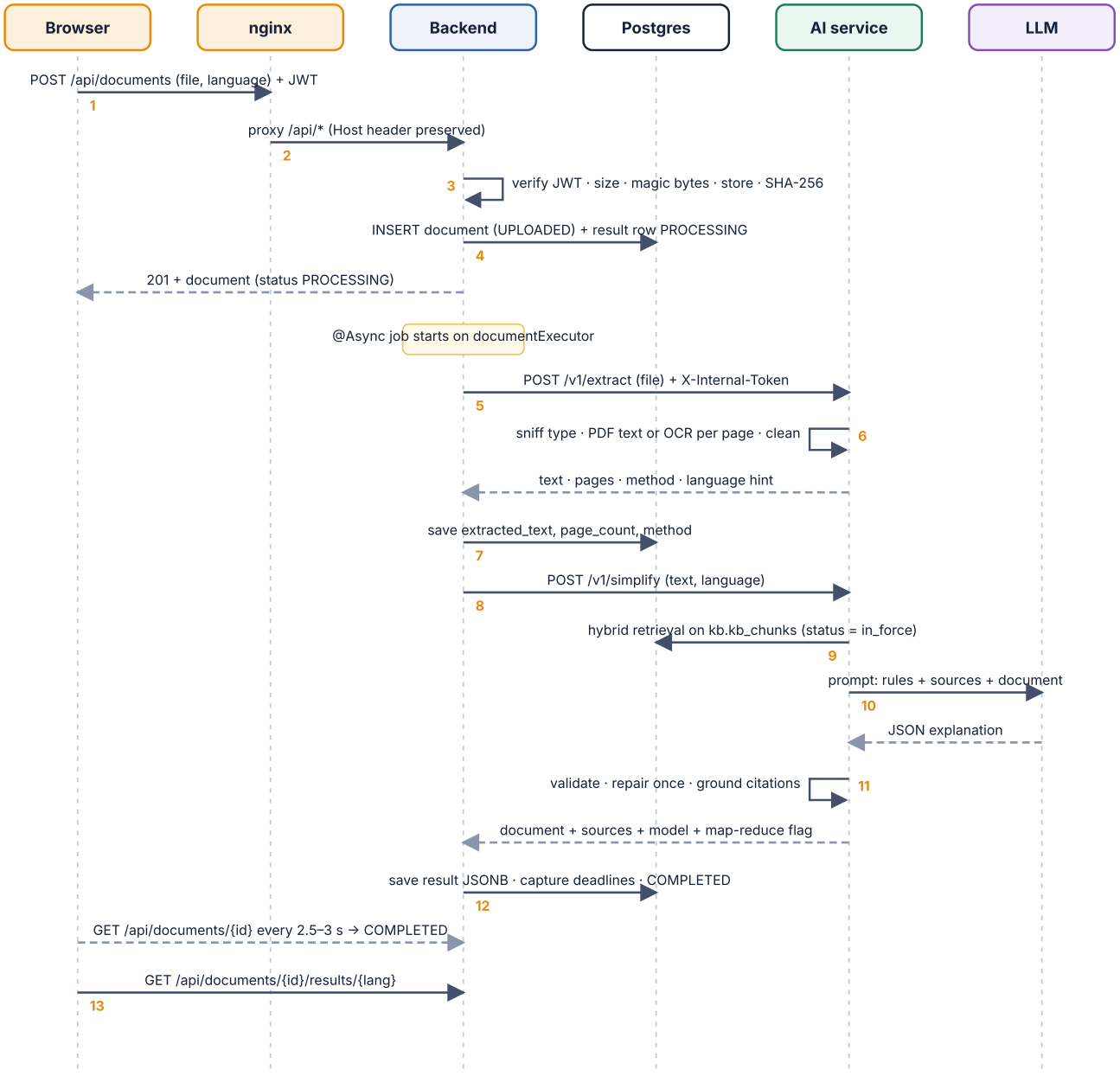


Figure 9.1 — From file to explanation. The browser gets its answer in milliseconds (the 201 reply after call 4) and then polls; the slow work happens in the background.

1. *(calls 1–2)* The browser sends the file and the chosen output language to `/api/documents` with the JWT in the `Authorization` header. nginx proxies it to the backend with the original `Host` header.
2. *(calls 3–4)* The backend authenticates, validates (size, magic bytes), stores the file under a random name, computes SHA-256, and saves the `documents` row plus a `processing_results` row in state PROCESSING. It returns **201 immediately**.

3. *(call 5)* A background job calls the AI service's `/v1/extract` with the stored file. The AI service decides page by page between the PDF text layer and OCR, cleans the text and returns it with metadata.
4. *(calls 7–8)* The backend saves the extracted text (so later languages and chat never re-run OCR) and calls `/v1/simplify`.
5. *(calls 9–11)* The AI service retrieves the relevant legal sources from `kb.kb_chunks`, builds the prompt, calls the LLM chain, validates and repairs the JSON, grounds the citations and returns the explanation.
6. *(calls 12–13)* The backend stores the result as JSONB, captures deadlines into their own table, and sets the status to COMPLETED. Meanwhile the browser polls the document every 2.5–3 seconds and then fetches the result for the chosen language.

What crosses which boundary: the file bytes go browser → backend → AI service (once); the extracted text goes backend ↔ AI service; the text and retrieved sources go to the LLM provider (the only data that leaves your infrastructure); the browser never talks to the AI service or the database.

9.2 A chat turn

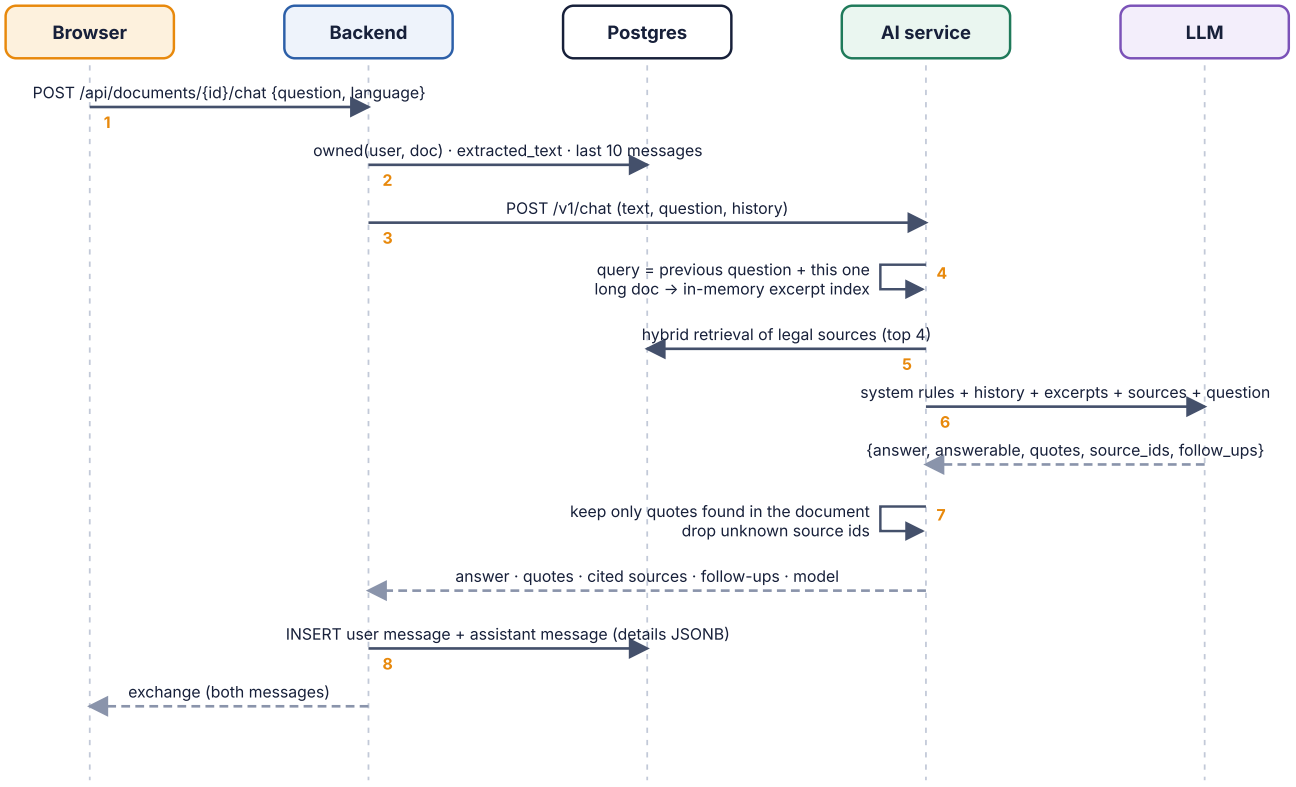


Figure 9.2 — One question and answer. The backend owns history and persistence; the AI service owns reasoning.

The backend sends the stored document text and the last 10 messages; the AI service replays the most recent 8 to the model, combines the previous question with the new one for retrieval, and returns an answer whose quotes were verified against the document and whose source ids were checked against what was retrieved. The AI service stores nothing; the backend writes both messages in one go after the AI call succeeds, so a failed call leaves no half-turn behind.

9.3 Knowledge-base ingestion at start-up

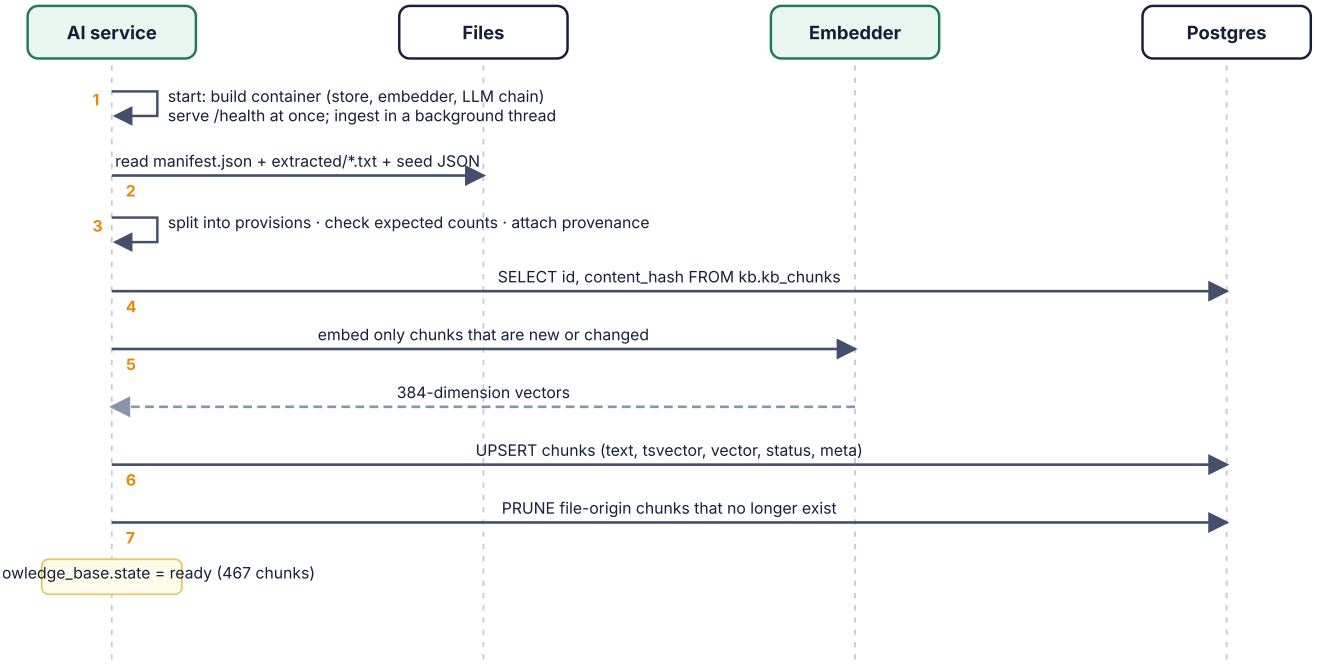


Figure 9.3 — Incremental, background ingestion. Only changed chunks are embedded.

Because chunks are keyed by id and carry a content hash, a restart costs a few queries instead of re-embedding everything. After a text correction (for example fixing an OCR digit in `extracted/*.txt`) only the affected chunks are re-embedded, and chunks that disappeared from the files are pruned.

9.4 Deadline reminders

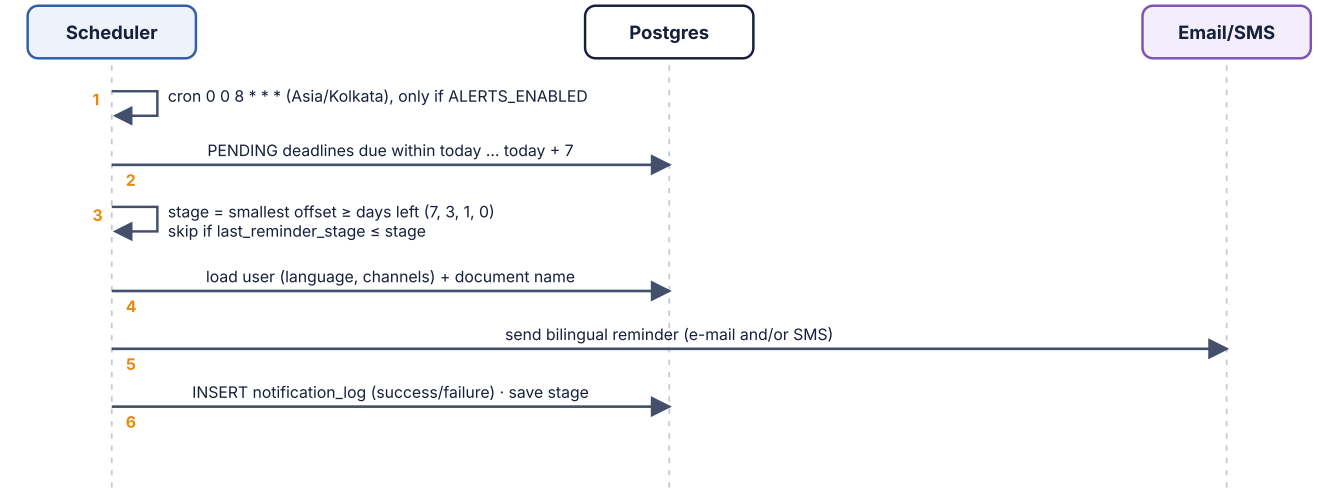


Figure 9.4 — The optional daily job (enabled with `ALERTS_ENABLED=true`).

Reminder stages default to 7, 3, 1 and 0 days. A deadline due in 5 days is in stage 7; the next run at 3 days moves it to stage 3. If the server was down for a day, the next run sends the reminder for the stage the deadline is *currently* in rather than skipping it, and `last_reminder_stage` guarantees the same stage is never sent twice. One user's failure never stops the batch, and every attempt, successful or not, is written to `notification_log`.

9.5 Draft generation

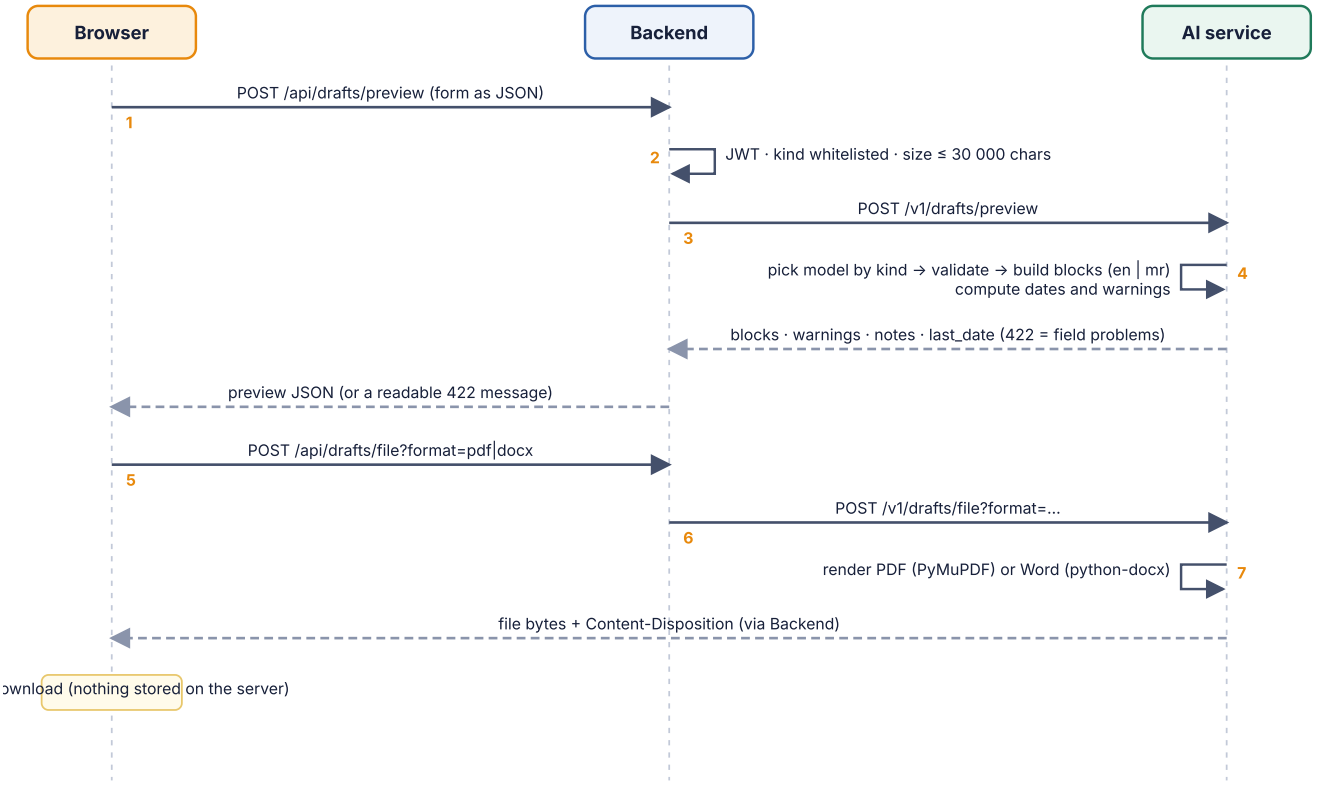


Figure 9.5 — Stateless: nothing typed into the form is stored; the file is built on request and downloaded through a blob URL.

9.6 Authentication

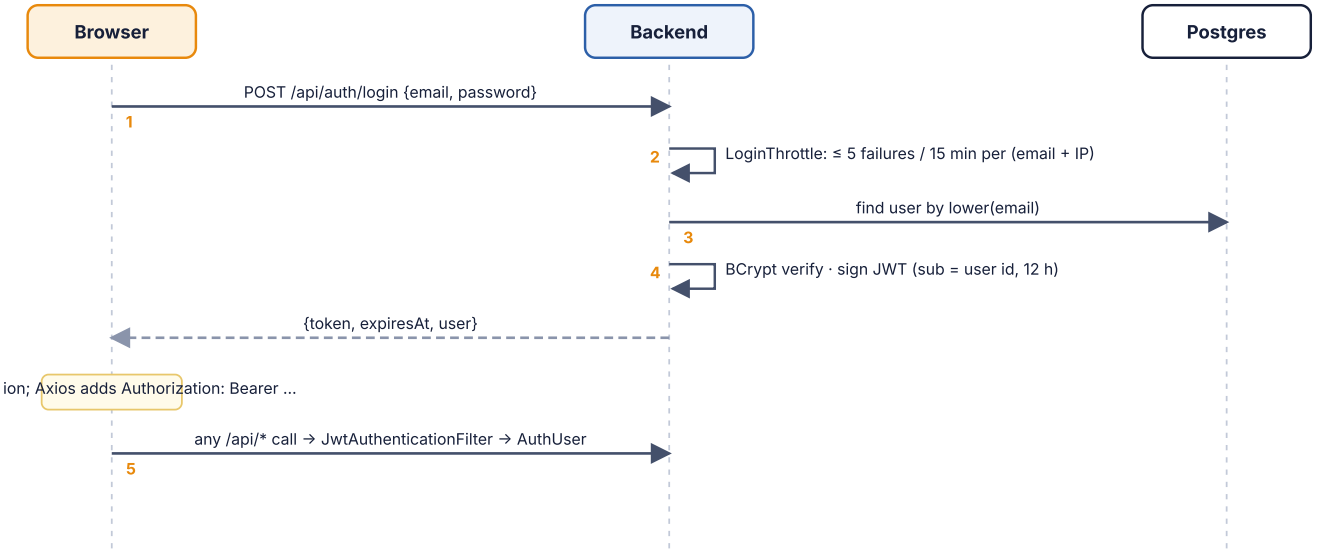


Figure 9.6 — Login and the stateless session.

9.7 Where each kind of data lives

Data	Stored in	Notes
Accounts, password hashes	Postgres `public.users`	BCrypt hash only; e-mail unique case-insensitively.
Original uploads	Docker volume `uploads`	Random file names under a per-user folder; served only through an authenticated endpoint.

Data	Stored in	Notes
Extracted text	Postgres `documents.extracted_text`	Reused for new languages and chat; deleted with the document.
Explanations	Postgres `processing_results.result` (JSONB)	One row per document and language.
Deadlines, reminder log	Postgres tables	Deadlines are extracted from the result into rows for querying and reminders.
Chat history	Postgres `chat_messages`	With structured details (quotes, sources, model).
Legal knowledge base	Postgres `kb.kb_chunks`	Rebuildable from the repository.
Models (embedding, reranker)	AI service image `/opt/models`	Baked in at build time so start-up is offline-safe.
Drafts	Nowhere	Generated and returned; the form never reaches a database.
JWT	Browser `localStorage`	Not stored on the server (stateless).

10 Functionality walk-through (interview view)

Each feature below is described the way you would explain it when asked “walk me through how this works”: what the user sees, what happens inside, the design decisions and trade-offs, edge cases that were handled, and the questions you should expect.

10.1 Registration, login and sessions

User sees: a registration form (name, e-mail, password, optional phone and preferred language) and a login form. **Inside:** Bean-Validation on the DTOs (valid e-mail, password 8–72 characters, phone in international format `+91…`), BCrypt(12) hashing, a case-insensitive unique e-mail index, a JWT returned with the profile, and the browser attaching it to every call.

- **Brute force:** `LoginThrottle` (5 failures per e-mail+IP in 15 minutes → 429). Same generic message for “unknown e-mail” and “wrong password” so accounts cannot be enumerated.
- **Expiry:** the JWT lives 12 hours; the front end logs out when it expires and on any 401.
- **Edge case fixed:** Spring needs an explicit `@Autowired` on the public constructor when a class has two constructors (one for tests with an injectable clock); the first run failed with “No default constructor found”.

Q Walk me through what happens when a request arrives with a JWT.

nginx proxies it to Spring. `JwtAuthenticationFilter` runs before the standard username/password filter, extracts the bearer token, verifies the HS256 signature and expiry with `JwtService`, and puts an `AuthUser` (id, email, role) in the security context. Controllers receive it with `@AuthenticationPrincipal`. If the token is missing or invalid, `RestAuthenticationEntryPoint` returns a JSON 401 and the front end logs the user out.

10.2 Upload, validation and extraction

User sees: a drop zone, a file chip, an output-language switch and a progress bar. **Inside:** section 5.4 (validation, atomic storage) and section 6.3 (extraction).

- **Accepted:** PDF, PNG, JPEG, TIFF, BMP, WEBP up to 25 MB, identified by content. **Rejected:** everything else with a specific message (a renamed `.exe` is refused even if called `.pdf`).
- **Mixed documents:** per-page choice between text layer and OCR; `method` is `digital`, `ocr` or `mixed`.
- **Marathi specifics:** broken text layers detected and OCR’d; Devanagari-safe cleaning and tokenization; Tesseract `mar+hin+eng`.
- **Failure modes handled:** password-protected PDF, zero-page PDF, unreadable scan (“No readable text found”), huge PDF (first 60 pages with a warning), AI service unreachable (retryable failure).

Q How does your OCR cope with a Marathi photo?

Tesseract runs with Marathi, Hindi and English models together, after grayscale conversion, autocontrast and up-scaling of small images. On synthetic noisy photos it recovered 98–100% of words; real photographs with shadows and perspective will be worse, so the UI warns when OCR was used and the knowledge-base sources flag OCR text. A natural improvement is deskewing, adaptive thresholding and an OCR confidence score shown to the user.

10.3 Bilingual explanation with citations

User sees: a title, a short summary, key points, an action checklist with priorities, a deadline timeline, legal references linked to numbered sources, sources with trust badges, and caveats, in English or Marathi, switchable without re-upload.

- **Structured output** is a Pydantic model (`SimplifiedDocument`): `document_type, title, summary, key_points, deadlines, required_actions, legal_references, caveats`.

- **Language switch** re-runs only the explanation step on the stored text; each language has its own result row and status.
- **Deadline discipline:** exact ISO date only when the document states or implies it; else a relative period such as “within 30 days of receiving this reply” plus the original sentence, so the user can verify.
- **Citations:** checked against retrieved sources; cited sources sorted first; trust badges and notes shown (section 4.4).

Q Show me an example of the system being careful about law that changed.

Maharashtra notified RTI Rules twice in 2026. The 12 June notification set fees at ₹30, ₹5 and ₹100; the 7 July notification replaced it with ₹10, ₹2 and ₹20 and appeal periods of 30 and 90 days. Both are in the knowledge base, but the 12 June one is marked `superseded` and excluded by every retrieval query. A test asserts that a fee question returns the 7 July rules and nothing from the superseded set.

10.4 Continuous Q&A about a document

User sees: a chat that remembers the conversation, suggested follow-ups, quotes from the document and cited sources, and a distinct “this document does not say” answer when applicable. **Inside:** sections 5.8, 6.5 and 9.2.

- History persists across reloads and is private to the user; the conversation can be cleared; at most 100 questions per document.
- Follow-ups are made retrievable by prepending the previous question.
- Quotes are verified against the document; invented quotes are dropped.
- `answerable: false` is an explicit contract, so “I could not find it” is not disguised as an answer.

Q How does the chat know what “it” refers to?

Two mechanisms. The last turns are sent to the model so it can resolve references. And retrieval uses the previous question plus the current one, so “what about the second one?” still finds the right passage in the document and the right law.

10.5 Deadline tracking and reminders

User sees: a Deadlines page grouped as Overdue, Next 7 days, Later, No fixed date and Done; a checkbox to mark items done; optional e-mail/SMS reminders in the user’s language. **Inside:** section 9.4.

- **Capture:** from the first processed language only, so translating does not duplicate rows. A deadline can have a date, a relative period, or both, plus the source sentence.
- **Reminder logic:** stage-based (7, 3, 1, 0 days, configurable), idempotent through `last_reminder_stage`, resilient to downtime, per-channel logging.
- **Time zone:** scheduled at 08:00 `Asia/Kolkata`, with “today” computed in IST so a late-night run does not shift the day.
- **Channels:** SMTP e-mail (Spring Mail) and Twilio SMS; each is enabled only when configured and when the user opted in.

Q What would break if you ran two backend instances?

Two things are per-instance: the login throttle (in memory) and the scheduler (it would send reminders twice, though `last_reminder_stage` limits duplicates). The fix is a shared store (Redis) for the throttle and a scheduler lock (for example ShedLock on a database row) or running the scheduler in one replica only, as the code comment says.

10.6 Trust: provenance, supersession and the knowledge base

Described in sections 7 and 8. In interview terms: the knowledge base is *data with metadata*, and the metadata is a product feature. Source type, current-as-of date, status, gazette reference, OCR flag, reviewer notes and verification state travel from the manifest, through the database, through the LLM prompt, to the badge the user sees.

- **Evidence of care:** hand-corrected OCR in the extracted gazette text; parser checks that fail loudly; separate notes for known staleness (2008 reprint, 2021 Rules changing consumer limits).
- **Honesty:** nothing is marked verified; seed summaries are labelled unverified.

10.7 Draft generator: RTI application, first appeal, fee-payment reply

User sees: a Drafts page with three cards, a form tailored to the chosen document, a language switch (English / Marathi), a preview on a "paper", warnings and filing notes, and buttons for PDF and Word. **Why it exists:** understanding a notice is only half the problem; the user then has to write something correct and on time.

Draft	Content and built-in logic
RTI application Annexure "A"	Follows the official form: applicant, office, subject, period, numbered description, post or in person (and type of post), BPL status with proof, fee paid by stamp/cash/draft/banker's cheque. Adds the ₹10 stamp box when relevant; if the applicant is below the poverty line, states that no fee applies (Section 7(5) proviso). **Warns** when the request exceeds 150 words (the Maharashtra rule says one subject, ordinarily within 150 words).
First appeal Annexure "B", s.19(1)	Inputs: appellate authority, PIO, application date, decision date (if any), grounds, information sought, fee mode, optional delay reason. **Computes the last date:** decision received + 30 days; or, if no reply, application date + 30 days (the PIO's time) + 30 days. **Warns** if the PIO still has time to reply (premature appeal), if the period has already ended and no delay reason is given, or if grounds contradict the dates. **Adds a "request to condone delay"** using the proviso to Section 19(1) when late. Each selected ground is written out citing its section (7(1)/7(2), 8/9/10, 7(3)(a)). The prayer asks for the information, for it free of charge if the time limit was missed (Section 7(6)), and for any other order; it deliberately does **not** ask for a penalty, which only the Information Commission can impose (Section 20) — a filing note says so.
Reply to a fee demand s.7(3)	Confirms payment (amount, mode, reference, date), asks for the information and notes the period between intimation and payment is excluded (7(3)(a)). If the user gives the page count and the amount demanded differs from pages × ₹2, it asks the PIO for the calculation and reserves the right to seek a review (7(3)(b)). **Warns** if the payment is less or more than the amount demanded.

HOW IT IS BUILT

- **Deterministic templates, not an LLM.** A legal form must say exactly what was entered. Templates exist in English and Marathi (with Devanagari numerals for dates and amounts generated by the template). The Marathi wording is based on the official Marathi forms and needs review by a Marathi speaker before real use.
- **Validation.** Pydantic models, selected by `kind` (one model per document type): required fields, length limits, control-character stripping, date sanity (not in the future, with one day of slack for time zones; letter after application; payment after letter), fee reference required for cash/draft/cheque. Errors come back as one readable line per field ("application_date: is in the future – check that the day and month are the right way round"), and the front end replaces field names by the labels on the form.
- **Rendering.** A neutral list of blocks (title, paragraph, rows, numbered list, signature…) feeds two renderers. The PDF uses PyMuPDF's HTML engine, which does proper Devanagari shaping with built-in fonts (no font installation), with explicit point widths because that engine ignores percentage table widths. The Word file uses python-docx with the *Mangal* font for Marathi (installed with Windows) and sets all four font slots so complex-script text renders correctly. User text is HTML-escaped in the PDF path.
- **Privacy.** Stateless end to end; the backend only whitelists the `kind`, limits the size, and proxies.

A BUG WORTH TELLING IN AN INTERVIEW

After release, the first-appeal and fee-reply forms failed with a wall of "RtiApplication... FeePaymentReply..." errors. Two causes: (1) the framework had stopped honouring the discriminated union, so the form was validated against every document type and all errors were reported together; and (2) the real problem, "application_date is in the future", came from a date check with no time-zone slack on a UTC server, and from day/month confusion in the browser's date field. Fixes: select the model by `kind` explicitly, accept dates one day ahead, block future dates in the picker, spell the chosen date out under the field, and map server field names to form labels. The lesson was that my earlier browser test had opened but never submitted those forms; the new end-to-end test submits all three.

Q Why templates and not the LLM for drafting?

Accuracy, auditability and availability. A legal document must contain exactly the entered facts and the correct provisions; an LLM can rephrase a date or invent a section. Templates are testable (there are tests for every branch), cheap, instant and work when the LLM is down. An LLM could still be added for optional "polish my description" help, with the template as the safe default.

10.8 Settings, privacy and account deletion

- Profile edits (name, phone, preferred language) and per-channel reminder toggles.
- **Delete my account** removes the user and, through `ON DELETE CASCADE`, every document, result, deadline, reminder log and chat message, and the user's upload directory. A test covers erasure.

10.9 Resilience and self-diagnosis of the LLM connection

Hosted LLMs fail in ordinary ways: a model is overloaded (503), a quota is reached (429), a model name is retired (404), a key is wrong (401). During development all four happened. The response was a **fallback chain**, clear error classification, and a diagnostic command. Section 6.6 has the design. The user-visible result: instead of "We couldn't finish this document (500)", a failed document says "The AI model rejected the API key... check `.env` and run `docker compose up -d`", and in the common overload case the user does not notice at all because a backup model answers.

10.10 Evaluation harness

A reproducible way to measure retrieval quality: 60 labelled questions (40 English, 17 Marathi, 3 out of scope), each with the provisions that contain the answer and, for some, required facts. `python -m app.evaluation retrieval` compares five methods and writes Markdown and JSON with bootstrap confidence intervals; `… answers` scores fact recall, citation hit/precision, numbers grounded in the cited text and refusal of out-of-scope questions when a real LLM is configured. Results are in section 13.

11 Security and privacy

The system handles personal legal documents. The security design assumes the browser is untrusted, the AI service is internal, and the LLM provider is a third party.

11.1 Threat → mitigation

Threat	Mitigation in this system
Reading another user's data	Ownership-scoped repository methods (`findByIdAndUserId`); user id taken from the verified JWT; misses return 404; integration tests try cross-user reads, deletes and chat.
Stolen or forged token	HS256 with a secret of at least 32 bytes (start-up fails otherwise); 12-hour expiry; front-end logout at expiry. Residual risk: token in `localStorage` (XSS) — see 4.3.
Password attacks	BCrypt cost 12; 5-failure throttle per e-mail+IP; same message for unknown user and wrong password, and a dummy hash is compared for unknown e-mails so timing does not reveal whether an account exists.
Malicious uploads	Content-based type check (magic bytes), size cap, random stored names, path-traversal check, atomic writes, non-root containers; PDFs are only parsed by PyMuPDF/Tesseract in the AI service, never executed or served inline from the upload directory.
Direct access to the AI service	Not published to the host; every route requires `X-Internal-Token` , compared in constant time; only the backend knows it.
SQL injection	JPA / parameterised queries everywhere. The one dynamically built SQL (IDF keyword search) only interpolates placeholders; the terms and weights are bound parameters, and terms come from a tokenizer that yields only letters and digits.
XSS / clickjacking	React escapes output; no raw HTML injection; CSP restricts scripts to the same origin; `X-Frame-Options: DENY` ; `nosniff` ; `Referrer-Policy: no-referrer` .
CSRF	No cookies are used for authentication, so there is no ambient credential to abuse; CORS lists only allowed origins.
Prompt injection via a document	The model's output is parsed against a strict schema and never executed; it cannot call tools or write data; citations and quotes are verified. A hostile document could still try to bias the summary, which is why the UI shows sources and caveats and says this is not legal advice.
Output injection into PDFs/Word	User text is HTML-escaped for the PDF renderer; Word receives plain runs; control characters are stripped by validation.
Secret leakage	Secrets live in `.env` (git-ignored; only `.env.example` is committed); API errors never echo keys; the AI service strips quotes/spaces from keys and error messages name the setting, not the value.
Denial of service	Upload size cap and nginx body limit; bounded async queue (overflow becomes a clear "server busy"); per-document chat limit; bounded form sizes for drafts; timeouts on every outbound call. (There is no global rate limiter — see limitations.)

11.2 Privacy considerations

- **What leaves the system.** The extracted text of a document (and the retrieved legal sources) is sent to the configured LLM provider. Hosted providers may log or, on free tiers, train on inputs. For sensitive material use a local model (`ollama`) or a paid tier with a no-training policy. This should be disclosed to users in a real deployment.
- **What does not.** Passwords (hashed), other users' data, and draft forms.
- **Data minimisation and erasure.** Users can delete a document or the whole account with cascading deletion of rows and files. Draft forms are never stored.
- **Planned improvement.** Detect and mask phone numbers, Aadhaar numbers and e-mail addresses before text is sent to the LLM, and show the user what was masked.

11.3 Container hardening

- Backend and AI service run as a non-root user (`uid 10001`); the backend and front end use multi-stage builds so build tools stay out of the runtime image; health checks on all services.
- Only port 3000 is published. Database, backend and AI service ports are internal to the Compose network.

Q What is the biggest security weakness today?

Honest answer: the JWT in `localStorage` and the missing global rate limiter. The first is mitigated by CSP but a refresh-token-in-HttpOnly-cookie design would be stronger; the second means an authenticated user could submit many expensive jobs (bounded only by the queue and the file-size cap). I would add per-user rate limits, a Redis-backed throttle, and an account-enumeration review of the registration response (it reports “e-mail taken”).

12 Deployment and operations

12.1 Compose file in plain words

Service	Image / build	Wiring
`postgres`	`pgvector/pgvector:pg16`	Volume `pgdata`; `db/init.sql` creates the `vector` extension on first start; `pg_isready` health check; the password is required (`POSTGRES_PASSWORD`).
`ai-service`	build `./ai-service` (Python 3.12-slim + Tesseract mar/hin/eng; models baked in)	Depends on healthy Postgres; environment for database URL, internal token, LLM provider/model/keys/fallbacks, embedding and reranker models; `expose: 8000` only; `host.docker.internal` mapped so a local Ollama can be reached.
`backend`	build `./backend` (Maven build stage, JRE 21 runtime)	Depends on healthy Postgres and *started* AI service; JWT secret and internal token required; `uploads` volume at `/data/uploads`; alert and mail/SMS settings; `expose: 8080`.
`frontend`	build `./frontend` (Node 22 build, nginx runtime)	Depends on healthy backend; publishes `${FRONTEND_PORT:-3000}:80`.

Start-up order: Postgres becomes healthy → AI service and backend start (the backend runs Flyway migrations) → frontend starts when the backend is healthy. The AI service loads models and the knowledge base in the background; `/health` reports progress.

12.2 Configuration (twelve-factor)

All settings are environment variables read into typed classes (`AppProperties` in Java, `Settings` in Python); `.env.example` documents them. The ones people change most:

Variable	Meaning
`POSTGRES_PASSWORD`, `JWT_SECRET`, `INTERNAL_TOKEN`	Required secrets (the stack refuses to start without them).
`LLM_PROVIDER`, `LLM_MODEL`, `OPENAI_API_KEY`, `OPENAI_BASE_URL`, `ANTHROPIC_API_KEY`	Primary model; `openai` works with any OpenAI-compatible endpoint (Groq, Gemini…).
`LLM_FALLBACK_MODELS`, `FALLBACK_LLM_PROVIDER/MODEL/API_KEY/BASE_URL`	Backup models on the same provider, then a second provider.
`EMBEDDING_PROVIDER`, `RERANKER`	`fastembed` / `cross-encoder` in production; `hash` / `lexical` for offline tests.
`ALERTS_ENABLED`, `ALERTS_CRON`, `MAIL_*`, `SMS_PROVIDER`, `TWILIO_*`	Optional reminder job and channels.
`FRONTEND_PORT`, `CORS_ALLOWED_ORIGINS`, `MAX_UPLOAD_MB`	Web settings.

12.3 Local development without Docker for every service

```
# database with pgvector (one container, or a local Postgres + CREATE EXTENSION vector)
docker run -d -p 5432:5432 -e POSTGRES_USER=nyayasetu -e POSTGRES_PASSWORD=nyayasetu -e POSTGRES_DB=nyayasetu
pgvector/pgvector:pg16
# AI service (needs tesseract-ocr + mar/hin packs)
cd ai-service && pip install -r requirements.txt && INTERNAL_TOKEN=dev uvicorn app.main:app --port 8000
# backend
cd backend && JWT_SECRET=<32+ chars> AI_INTERNAL_TOKEN=dev mvn spring-boot:run
# frontend (proxies /api to :8080)
cd frontend && npm install && npm run dev
```

12.4 Images, size and rebuilds

- The AI service image is the largest (several GB): Python packages (ONNX runtime, fastembed, PyMuPDF…) plus about 1 GB of baked models. A complete set of images is roughly 8–10 GB.
- The Dockerfile installs the ML libraries and downloads the models *before* the other requirements, so editing `requirements.txt` or the code does not re-download the models. Without this ordering, each rebuild left a multi-gigabyte unused layer behind (a real disk-space problem on a Windows laptop).
- Housekeeping: `docker image prune` and `docker builder prune` after rebuilds; never `docker volume prune` (it deletes `pgdata`). On Windows with WSL2, freed space returns to drive C: only after the Docker disk file is compacted.

12.5 Continuous integration

GitHub Actions runs three jobs on every push: **ai-service** (installs Tesseract with Marathi/Hindi packs, starts a pgvector Postgres and runs pytest including the real-OCR and real-pgvector tests), **backend** (`mvn verify` against Postgres) and **frontend** (`npm ci && npm test && npm run build`).

12.6 Observability and diagnostics

- `/actuator/health` (backend, including an AI-service indicator) and `/health` (AI service: database, chunk count, ingestion state, models, LLM chain).
- `docker compose exec ai-service python -m app.llm.check` → which models answer; `--list-models` → names the keys can use.
- Logs name the failing step and model on every fallback (“LLM openai:… failed (llm_overloaded)”); no secrets are logged.

13 Testing and evaluation (with real results)

13.1 Automated tests

Code base	Tests	What they cover
AI service (pytest)	**101**	Text cleaning and Devanagari tokenizer; PDF and image extraction including **real Tesseract OCR** on generated Marathi/English images; garbled-text detection; chunking; hybrid retrieval, RRF and the reranker; **real pgvector** retrieval and idempotent re-ingestion; official-source parsers (rule counts, supersession chain, July 2026 fees, Consumer Protection sections, provenance reaching the prompt); simplifier (JSON repair, hallucinated-source removal, map-reduce); chat (history, quote verification, unanswerable); LLM fallback chain against a scripted OpenAI-style server (503, 429, 401, 404, chain order, batch use); error classification; evaluation metrics; the draft generator (every branch of the templates, validation, date rules, PDF/Word output, OCR of the Marathi PDF to prove glyph shaping, HTML escaping, API errors).
Backend (JUnit + MockMvc)	**27**	Registration/login validation, JWT, throttle, file-type sniffing and storage, **full-stack tests on a real PostgreSQL with Flyway** (upload → process → results → deadlines; users cannot see each other's documents; AI failure is reported and retryable; non-document uploads rejected; account erasure; persisted private chat; chat requires extracted text; drafts proxy), and the reminder-stage logic.
Frontend (Vitest)	**11**	Date formatting and deadline grouping; draft form rules: required fields, hidden fields, payload building, future-date detection, field-label mapping.
Browser end-to-end (Playwright, scripted)	—	Run during development with the real stack (backend jar, AI service, Vite) and a scripted stand-in for the LLM: registration, upload of PDFs/photos, summary, Marathi, chat, deadlines, provenance badges, and all three draft forms with downloads. These scripts are not part of CI.

> **WHAT THE TESTS DELIBERATELY DO NOT PROVE**
>
> The real embedding model, the real reranker and any real LLM cannot be exercised in the CI environment's offline mode. They were exercised separately: the retrieval evaluation below ran with the real models on a local machine; answer quality with a real LLM was checked by hand on sample documents, not scored.

13.2 Retrieval evaluation: method

- **Question set:** 60 questions in `ai-service/eval/questions.jsonl`: 40 English and 17 Marathi answerable questions across RTI procedure, consumer law and other laws, 12 of them phrased informally ("I filed an RTI and got no reply..."), plus 3 out-of-scope questions (not scored for retrieval).
- **Gold labels:** for each question, the knowledge-base provisions that contain the answer, assigned by reading the source text (not by looking at what the retriever returned). A test fails if a label points at a missing or superseded provision.
- **Metrics: Hit@k** = a gold provision is among the top k results; **MRR** = mean of 1/rank of the first gold provision (depth 10); 95% bootstrap confidence interval; and "superseded text returned".
- **Ablation:** each method adds one component to the previous one.

13.3 Retrieval evaluation: results

57 scored questions, real models (`paraphrase-multilingual-MiniLM-L12-v2` embeddings and `jina-reranker-v2-base-multilingual`), 467 chunks, run on 14 October 2026.

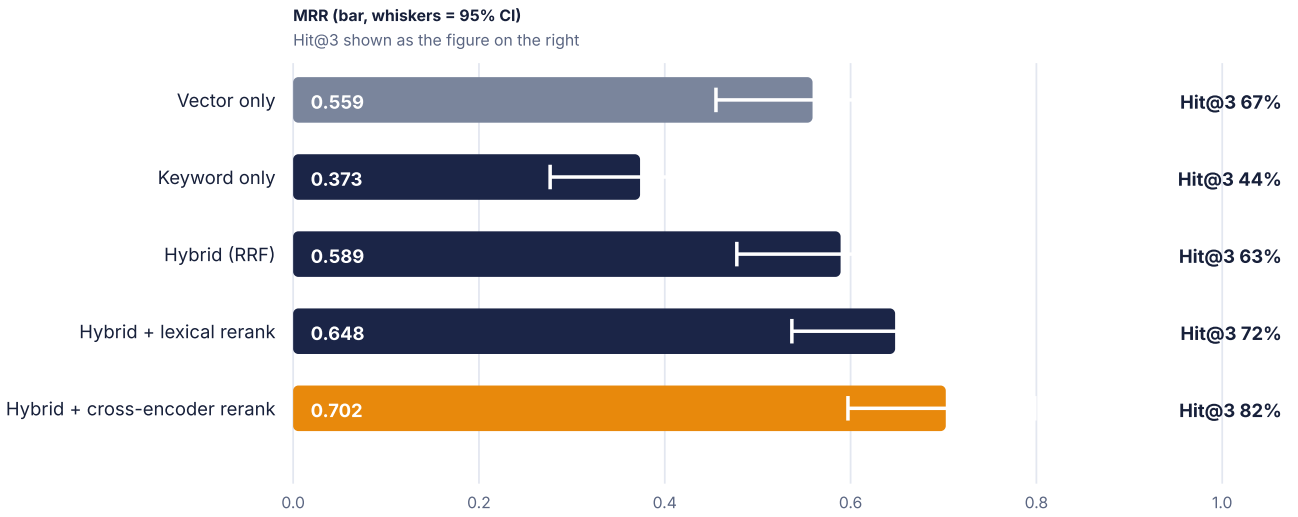


Figure 13.1 — Retrieval quality of the five methods (57 questions, real models). Source: eval-results/retrieval-2026-10-14.json.

Method	Hit@1	Hit@3	Hit@5	MRR (95% CI)	Superseded returned
vector only	42%	67%	72%	0.559 (0.46–0.66)	0%
keyword only (FTS)	26%	44%	51%	0.373 (0.28–0.48)	0%
hybrid (RRF)	49%	63%	74%	0.589 (0.48–0.70)	0%
hybrid + lexical rerank	54%	72%	81%	0.648 (0.54–0.75)	0%
hybrid + cross-encoder rerank (jina)	60%	82%	84%	0.702 (0.60–0.80)	0%

WHAT THE RESULTS SHOW

- Each added component helped: **MRR 0.559 → 0.589 → 0.648 → 0.702** from vector-only to the full pipeline, and the correct provision was in the top three for **82%** of questions with the cross-encoder.
- **Superseded law was never returned** by any method (0%).
- **English is stronger than Marathi** (full pipeline MRR 0.744 vs 0.605). Several Marathi misses are questions whose answer sits in the noisy scanned Marathi Act.
- **Direct questions beat informal ones** (MRR 0.766 vs 0.465).
- The confidence intervals are wide (about ±0.1) because there are only 57 questions: small differences between neighbouring methods should not be over-interpreted.

THE EVALUATION FOUND A REAL BUG

The keyword-only score (MRR 0.373) was far below what the same keyword code had scored in an earlier offline run (0.695). The cause: the in-memory store weighted terms by rarity, but the Postgres query used `ts_rank_cd`, which treats “how”, “many” and “days” the same as “poverty” or “penalty”. Reproducing the gap against the local Postgres confirmed it, and the fix (IDF-weighted scoring, section 7.3) raised keyword-only MRR from 0.373 to 0.661 and hybrid from 0.480 to 0.594 in the offline-embedder run. The fixed system has not yet been re-measured with the real models, so the table above shows the *before-fix* numbers for the keyword and hybrid rows.

13.4 Answer-quality evaluation (built, partly run)

The harness also scores answers from a real LLM: required facts present (fact recall), whether a relevant provision was cited (citation hit and precision), whether every number in the answer appears in the cited text, and whether out-of-scope questions are declined. The scoring logic is unit-tested with a stub model. It was not run at scale against a real LLM for this report, so no answer-quality numbers are claimed. The string checks flag likely errors but cannot prove an answer correct; a human reviewer should read a sample.

13.5 Other measurements

- **OCR:** 98–100% word recall on three synthetic noisy phone-photo images (English and Marathi).
- **Draft PDFs:** Marathi PDF shaping verified by rendering the PDF to an image and OCR-ing it with the Marathi model, plus visual inspection.

Q How do you know your RAG pipeline works?

I measured it. I wrote 57 scored questions with gold provisions, compared five retrieval methods with confidence intervals, and found that hybrid plus a cross-encoder reranker reached MRR 0.70 and top-3 hit rate 82%, with zero superseded results. The evaluation also exposed a defect in my keyword ranking, which I then fixed and added a regression test for. I am explicit about the limits: a small set, labels I assigned myself, and answer quality not yet scored at scale.

Q Why MRR and Hit@k rather than accuracy?

Retrieval returns a ranked list, and what matters downstream is whether the right provision is near the top, because only the top few sources go into the LLM prompt. Hit@k answers “is it in the prompt?”, MRR rewards ranking it higher.

14 Problems faced and how they were solved

Interviewers like specific stories. Each row is a real problem from the build: symptom, root cause, fix and lesson.

Problem	Root cause	Fix	Lesson
Backend refused to start: “non-empty schema public”	The AI service created `kb_chunks` in `public` before Flyway’s first run.	Knowledge base moved to its own `kb` schema.	Give each service its own schema; start-up order must not matter.
Uploads from the backend corrupted at the AI service	The JDK HTTP client attempted an HTTP/2 cleartext upgrade that uvicorn mishandled for multipart bodies.	Force HTTP/1.1 in the client.	Protocol negotiation can break “working” requests.
Browser POSTs got 403	The proxy rewrote `Host`, so Origin and Host differed and Spring’s CORS check failed.	nginx `proxy_set_header Host $http_host`; Vite `changeOrigin: false`.	Same-origin design is simpler than CORS configuration.
Marathi gazette text unreadable though PDFs looked fine	A broken text layer (doubled vowel signs).	`looks_garbled()` → OCR fallback per page.	“Has text” does not mean “has good text”.
Wrong rule numbers from OCR (“१९९.” instead of “११.”)	OCR digit confusion in Devanagari.	Reviewable extracted text files, hand corrections, sequential-number parsing, expected-count checks.	Put a human-checkable step between OCR and the database.
Fees quoted wrongly (₹30 vs ₹10)	Two notifications in one month; the older was loaded as current.	Status field and supersession chain; retrieval excludes superseded text; a test pins the current fees.	Law has versions; model them.
Citations to non-existent sources	LLMs can cite ids that were never provided.	Grounding pass drops unknown source ids; chat quotes are verified against the document.	Validate model output; do not trust it.
Python `\w` broke Marathi words	Devanagari vowel signs are not word characters in Python’s regex.	Custom tokenizer that keeps matras; language detection by script ratio.	Test with the real script, not just Latin text.
LLM calls failing: 401, 404, 503 “high demand”	Wrong key type, retired model name, overloaded provider.	Coded error messages, key cleaning, fallback chain, `llm.check` and `--list-models`.	Treat the LLM as an unreliable dependency; design for failure and diagnosability.
Keyword search weaker on Postgres than in memory	`ts_rank_cd` ignores term rarity.	IDF-weighted scoring from `ts_stat`; regression test.	Measure; the evaluation caught what unit tests could not.
Drafts failed with a long list of errors	Framework no longer honoured the discriminated union; date check lacked time-zone slack; day/month confusion in date input.	Model chosen by `kind`; one-day tolerance; future dates blocked; chosen date spelled out; field labels in errors.	Test the whole path through the UI, including submission.
Docker disk usage grew 8–10 GB per run	Model download layer invalidated whenever `requirements.txt` changed.	Reordered Dockerfile layers; documented pruning.	Order layers from least to most frequently changed.

Problem	Root cause	Fix	Lesson
“No default constructor” for a security bean	Two constructors (one for tests) and no `@Autowired` marker.	Annotate the public constructor.	Testability hooks can change how the framework wires beans.

15 Limitations and future work

15.1 Known limitations (say these first in a viva)

- **Not legal advice, and not verified law.** The knowledge base is not human-verified (all `verified_by` fields are empty). The Marathi RTI Act is OCR text; the English Act is a 2008 reprint that lacks later amendments; the other laws are short unverified summaries.
- **Marathi retrieval is weaker than English** (MRR 0.61 vs 0.74), partly because the Marathi source is noisy; a multilingual query step that also searches in English would help.
- **Answer quality depends on the LLM.** A small or non-Marathi-focused model can produce weak or incomplete Marathi, or miss a deadline. Answer-level scoring has not been run at scale.
- **OCR on real photographs** (shadows, perspective, handwriting) will be worse than the clean tests; there is no confidence display or deskew step.
- **Single-instance assumptions:** in-memory login throttle and a scheduler that must run in one replica.
- **Privacy:** document text is sent to a third-party LLM; no PII masking yet.
- **Jurisdiction:** Maharashtra and central RTI rules only; drafts follow the Maharashtra forms.
- **Jobs are in memory:** a restart loses queued jobs (they are marked failed and can be retried).
- **No refresh tokens, no global rate limiting, registration reveals existing e-mails.**

15.2 Roadmap

Idea	Why and how
Human verification workflow	Fill `verified_by/at` after a reviewer checks each source; show it in the UI (already supported).
More official texts	Consumer Jurisdiction Rules 2021, Central RTI Rules 2012, a current English RTI Act, Limitation Act, NI Act, BNS/BNSS.
Cross-language retrieval	Translate or expand Marathi queries to English and merge results.
PII masking before the LLM	Regex/NER for phone, Aadhaar, e-mail; show what was masked.
Voice and accessibility	Read-aloud and voice input in Marathi (browser speech APIs), larger-type mode.
Feedback loop	“This answer is wrong” button that stores the case and grows the evaluation set.
Case tracker	One RTI journey (application → reply → appeal) with status and deadlines linked.
Scale-out	Redis throttle, scheduler lock, a durable job queue, refresh-token auth, per-user rate limits.
Streaming answers	Token streaming in chat for responsiveness.

16 Interview question bank

Short model answers to questions that are likely in a viva or a technical interview on this project. Longer explanations are in the sections referenced.

16.1 Big picture

Q Describe the project in 30 seconds.

NyayaSetu helps ordinary people understand legal and RTI documents in English or Marathi. You upload a PDF or a phone photo; the system reads it (with OCR if needed), retrieves the relevant sections of Indian law from a knowledge base, and uses an LLM to produce a plain-language summary, deadlines, next steps and cited sources. You can ask follow-up questions, get reminders before deadlines, and generate ready-to-file RTI drafts. It runs as four containers: React front end, Spring Boot gateway, a FastAPI AI service, and PostgreSQL with pgvector.

Q What is the hardest technical problem in it?

Trustworthiness. Getting an LLM to produce fluent text is easy; making sure it is grounded, current and verifiable is the work: OCR on Marathi scans, a knowledge base with provenance and supersession, retrieval that finds the right provision, output validation and citation grounding, and an evaluation that measures it.

Q What is your own contribution versus libraries?

The libraries do OCR, embeddings, PDF parsing and LLM transport. My work is the architecture, the legal-text parsers and manifest, the hybrid retrieval and its IDF keyword scoring, the prompt and validation/grounding layers, the fallback chain, the deadline and reminder logic, the draft generator with its legal rules, the security design, and the test and evaluation harness.

16.2 Architecture and design

Q Why a Java gateway in front of a Python service?

Each language does what its ecosystem is best at, and the browser never reaches the AI code. The gateway gives authentication, ownership checks, validated storage, job orchestration and graceful degradation; the AI service stays stateless and replaceable. The cost is one extra network hop and two code bases to maintain.

Q Why asynchronous processing with polling?

OCR plus an LLM call can take tens of seconds. Holding an HTTP request open that long is fragile. The API returns immediately; a background job updates a status row; the UI polls. It is simple and robust. Jobs are kept in memory, so a restart loses queued work (marked failed and retryable); a durable queue would be the next step.

Q How is the AI service protected?

It is not published outside the Docker network and requires a shared secret header verified in constant time. Only the backend knows the secret.

Q What is idempotency in this system and where do you use it?

Starting a job twice must not do the work twice: `beginProcessing` refuses a second concurrent job and a repeat of a completed one unless forced. Knowledge-base ingestion is idempotent through content hashes. Reminders are idempotent through `last_reminder_stage`.

Q Why JSONB for results but tables for deadlines?

The explanation is a document consumed whole, so JSONB allows the AI schema to change without migrations. Deadlines are queried, filtered, sorted and used by the scheduler, so they belong in a normal table with indexes (including a partial index on pending deadlines).

16.3 AI and RAG

Q Explain RRF.

Reciprocal Rank Fusion scores a document as the sum over rankings of 1/(60 + rank). It needs only ranks, so lists with incomparable scores (cosine similarity and a keyword score) can be merged, and a document ranked well in both rises.

Q Bi-encoder versus cross-encoder?

A bi-encoder embeds query and passage separately, so passages can be pre-computed and searched fast, but it compares only two vectors. A cross-encoder reads query and passage together and judges relevance directly: more accurate, but it must run per pair, so I use it only on the 16 fused candidates.

Q Why chunk the knowledge base, and at what size?

Embeddings represent a limited span well, and the prompt has limited room. Entries (a section or rule) are split into 1 200-character chunks with 150 overlap; most sections fit in one chunk. Each chunk keeps the provision id, so retrieval results can be grouped back to a citation.

Q What if the retriever returns nothing relevant?

The prompt allows an empty source list; the model then explains from the document only and puts uncertainty in caveats. In chat, `answerable: false` is returned when neither the document nor the sources contain the answer.

Q How do you handle a long document?

Map-reduce: split into chunks, condense each in parallel into fact-preserving notes, and summarise from the notes. Chat uses a per-document index to fetch only relevant excerpts.

Q Temperature and why 0.1?

Low temperature makes extraction of dates, amounts and sections more deterministic. This is not creative writing.

Q What do you do about the model returning invalid JSON?

Tolerant extraction of the first balanced JSON object, Pydantic validation with lenient normalisation, one repair retry that shows the model its error, then a clear 502 if it still fails.

Q How would you evaluate answer quality, not just retrieval?

A labelled question set with required facts, scoring fact recall, whether the answer cites a relevant provision, whether numbers appear in the cited text, and refusals on out-of-scope questions. Plus human review of a sample, because string checks cannot prove correctness. The harness exists; the large-scale run is future work.

16.4 Backend and data

Q How do you stop path traversal and fake file types?

Type is decided from the first bytes; stored names are random UUIDs; every resolved path is checked to stay under the storage root; files are written to a temp file and moved atomically.

Q Why is chat `ask()` not `@Transactional`?

It calls a slow external service. A transaction would hold a database connection for the whole call and exhaust the pool under load. Persistence happens in short steps before and after the call.

Q What is Flyway and why use it?

A schema-migration tool: versioned SQL files (V1, V2…) applied in order and recorded in a history table. It makes the database schema reproducible and reviewable, instead of letting the ORM guess.

Q How do you delete a user's data completely?

`ON DELETE CASCADE` on every child table (documents, results, deadlines, reminders, chat) plus deleting the user's upload directory.

16.5 Frontend

Q How does the UI show Marathi correctly?

Marathi strings are UTF-8 in the bundle; the browser shapes the text. The UI language and the AI output language are independent settings. Draft PDFs are shaped server-side by MuPDF's engine; Word files rely on the Mangal font.

Q How do you avoid re-render and polling waste?

`usePolling` runs only while a job is active and the tab is visible, always calling the latest callback through a ref.

16.6 Testing and process

Q Give an example of a bug your tests caught and one they missed.

Caught: OCR misreading rule numbers, via the expected-count parser checks; a hallucinated source id, via the grounding test. Missed: the first-appeal and fee-reply form errors; my browser test opened those forms but never submitted them. After that I added end-to-end submission tests for all three drafts and API tests for every document type in both languages.

Q What would you do differently?

Start the evaluation harness earlier (it found the keyword-scoring defect), build a human-verification workflow for the legal texts from the beginning, choose a durable job queue, and test with real phone photographs sooner.

Q How would you scale this to a million users?

Move jobs to a durable queue with worker pods; run several stateless backend and AI replicas behind a load balancer; shared Redis for throttling and caching; a scheduler lock; object storage instead of a local volume; partition or move the vector index if the corpus grows; per-user rate limits and cost controls on LLM calls; a read replica for history.

16.7 Product and ethics

Q Is this legal advice?

No, and the system says so in the UI and every prompt. It explains documents and points to sources. For important decisions users should consult a lawyer or the District Legal Services Authority.

Q What are the risks of an AI explaining legal documents?

Hallucinated facts, outdated law, mistranslation and overconfidence. The design addresses them with grounding, provenance and supersession, citation checks, caveats and visible uncertainty, but it cannot remove them, which is why evaluation and human verification are on the roadmap.

Q What data does a third-party LLM see?

The extracted document text and the retrieved legal sources. Users should be told this; for sensitive documents a local model or a no-training plan is advisable, and PII masking is a planned feature.

A Appendix

A.1 Numbers worth remembering

Item	Value
Services / published ports	4 containers; only the front end is published (3000 → 80)
Upload limit · file types	25 MB · PDF, PNG, JPEG, TIFF, BMP, WEBP (by content)
JWT lifetime · password · throttle	12 h · BCrypt cost 12, 8–72 chars · 5 failures / 15 min per e-mail+IP
Async job pool	2 core / 4 max threads, queue 100; one retry after 2 s for transient errors
OCR	Tesseract mar+hin+eng · 200 DPI (300 for KB) · up to 60 pages
Direct-prompt limit · map-reduce	14 000 characters · chunks ≈ 5 400, ≤ 24, 4 in parallel
Knowledge base	318 entries → 467 chunks · 1 200-character chunks, 150 overlap · vector(384)
Retrieval	20 vector + 20 keyword candidates → RRF (k = 60) → 16 → rerank → top 6 (chat 4)
LLM	temperature 0.1 · max tokens 4096 · timeout 120 s · 1 retry per model when fallbacks exist
Chat	10 messages loaded · 8 replayed · each ≤ 1 500 chars · ≤ 100 questions per document · ≤ 2 verified quotes
Reminders	7, 3, 1, 0 days · daily 08:00 Asia/Kolkata
RTI periods used by the drafts	PIO reply 30 days (s.7(1)); first appeal 30 days (s.19(1)); second appeal 90 days (s.19(3))
Maharashtra fees (rules of 7 July 2026)	₹10 application · ₹2 per page · ₹20 appeal
Tests	101 AI service · 27 backend · 11 front end
Best retrieval result	MRR 0.702 · Hit@3 82% · 0% superseded text

A.2 Glossary

Term	Meaning
RTI	Right to Information Act, 2005: citizens may request information from public authorities.
PIO / SPIO / CPIO	(State / Central) Public Information Officer: the officer who answers RTI applications.
First Appellate Authority	Senior officer in the same authority who hears the first appeal (Section 19(1)).
Information Commission	Central or State body for second appeals and complaints; the only body that can impose the Section 20 penalty.
Deemed refusal	No reply within the time limit is treated as a refusal (Section 7(2)).
Section 8 / 9 / 10	Exemptions from disclosure · grounds of copyright · severability (release the non-exempt part).
Superseded	Replaced by a later rule or notification. Kept for history, excluded from retrieval.
Gazette	The government's official publication of Acts, Rules and notifications.
RAG	Retrieval-augmented generation: retrieve relevant text, then let the model answer using it.
Embedding	A vector representing the meaning of a text; similar meanings give nearby vectors.

Term	Meaning
HNSW	Hierarchical Navigable Small World: an approximate nearest-neighbour index used by pgvector.
RRF	Reciprocal Rank Fusion: merge ranked lists by summing 1/(k + rank).
Cross-encoder	A model that scores a (query, passage) pair jointly; used to rerank.
IDF	Inverse document frequency: weights rare words more than common ones.
MRR / Hit@k	Mean reciprocal rank of the first correct result · share of questions with a correct result in the top k.
JWT	JSON Web Token: a signed token carrying the user's identity.
Flyway	Database migration tool that applies versioned SQL scripts.
OCR	Optical character recognition: turning an image of text into text.

A.3 Useful commands

```
docker compose up -d                                  # start (add --build only when code changed)
docker compose up -d --build ai-service               # rebuild one service
docker compose exec ai-service python -m app.llm.check               # which LLMs answer
docker compose exec ai-service python -m app.llm.check --list-models # model names the keys can use
docker compose exec ai-service python -m app.evaluation retrieval    # retrieval evaluation
docker compose logs --tail 50 ai-service              # recent AI-service logs
docker image prune -f && docker builder prune -f      # reclaim disk (never prune volumes)
# tests
cd ai-service && TEST_DATABASE_URL=postgresql://… pytest -q
cd backend    && TEST_DB_URL=jdbc:postgresql://localhost:5432/nyayasetu_test mvn test
cd frontend   && npm test
```